

Data Streaming

Thoai Nam

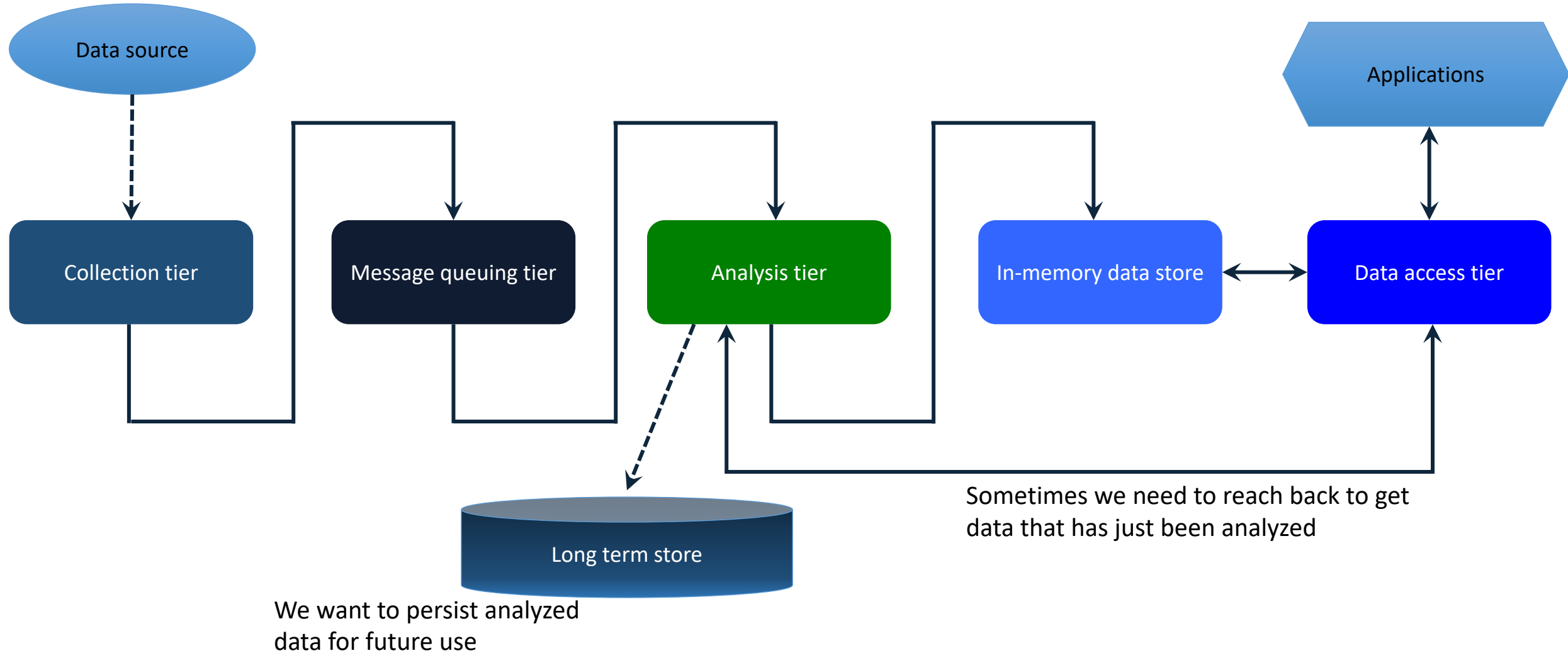
High Performance Computing Lab (HPC Lab)
Faculty of Computer Science and Engineering
HCMC University of Technology

Classification of real-time systems

Classification	Examples	Latency measured in	Tolerance for delay
Hard	Pacemaker, anti-lock brakes	Microseconds–milliseconds	None—total system failure, potential loss of life
Soft	Airline reservation system, online stock quotes, VoIP (Skype)	Milliseconds–seconds	Low—no system failure, no life at risk
Near	Skype video, home automation	Seconds–minutes	High—no system failure, no life at risk

Data streaming???

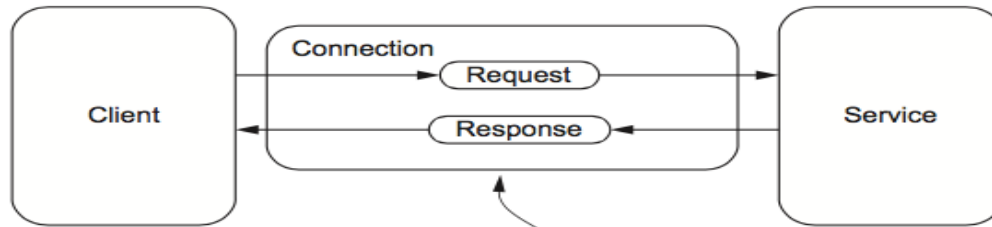
Data streaming architecture



Data collection: data ingestion

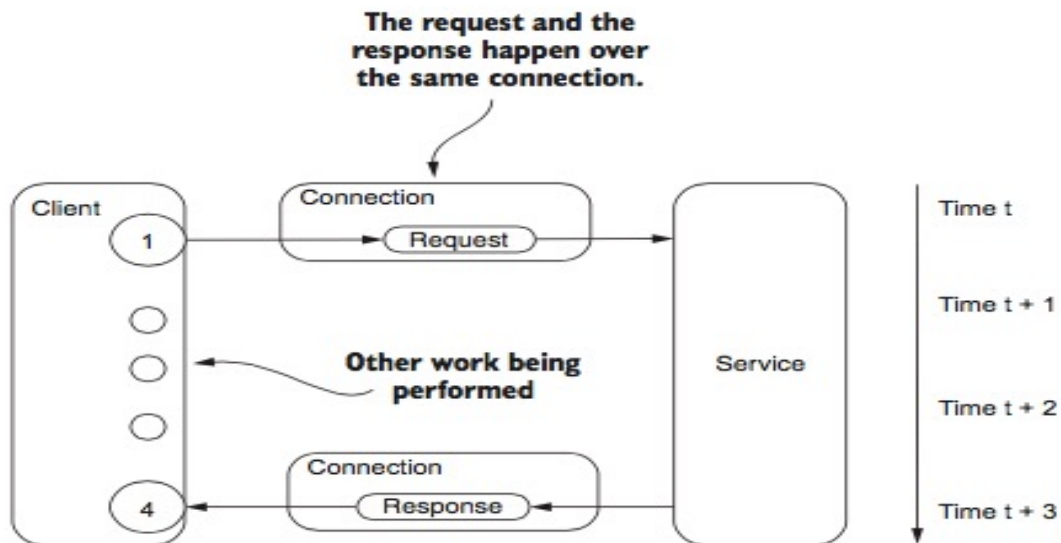
- Request/response
- Publish/subscribe
- One-way
- Request/acknowledge
- Stream

Request/response

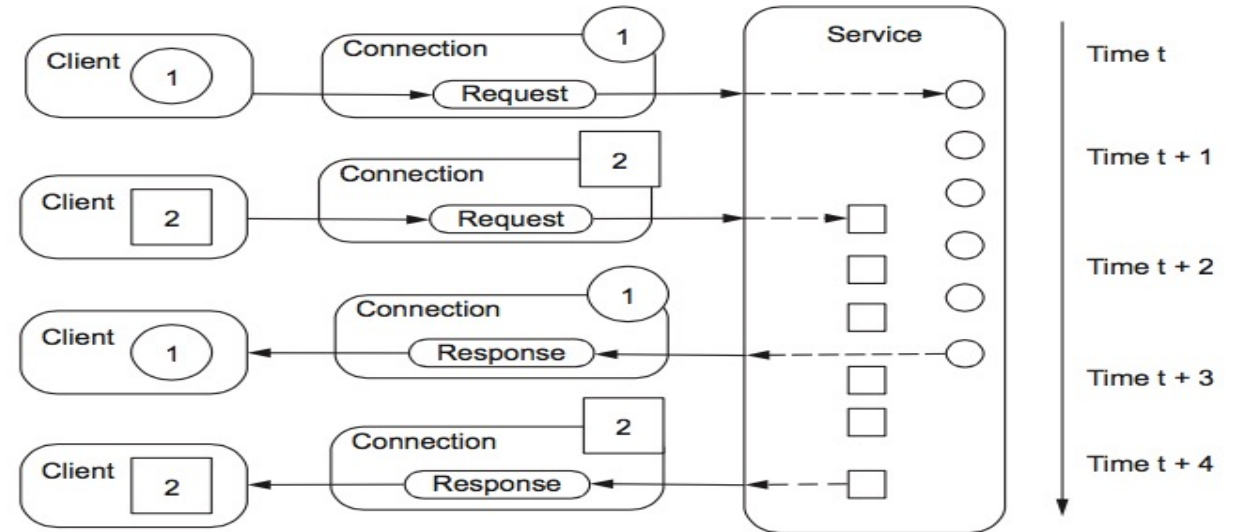


(a) Sync RR

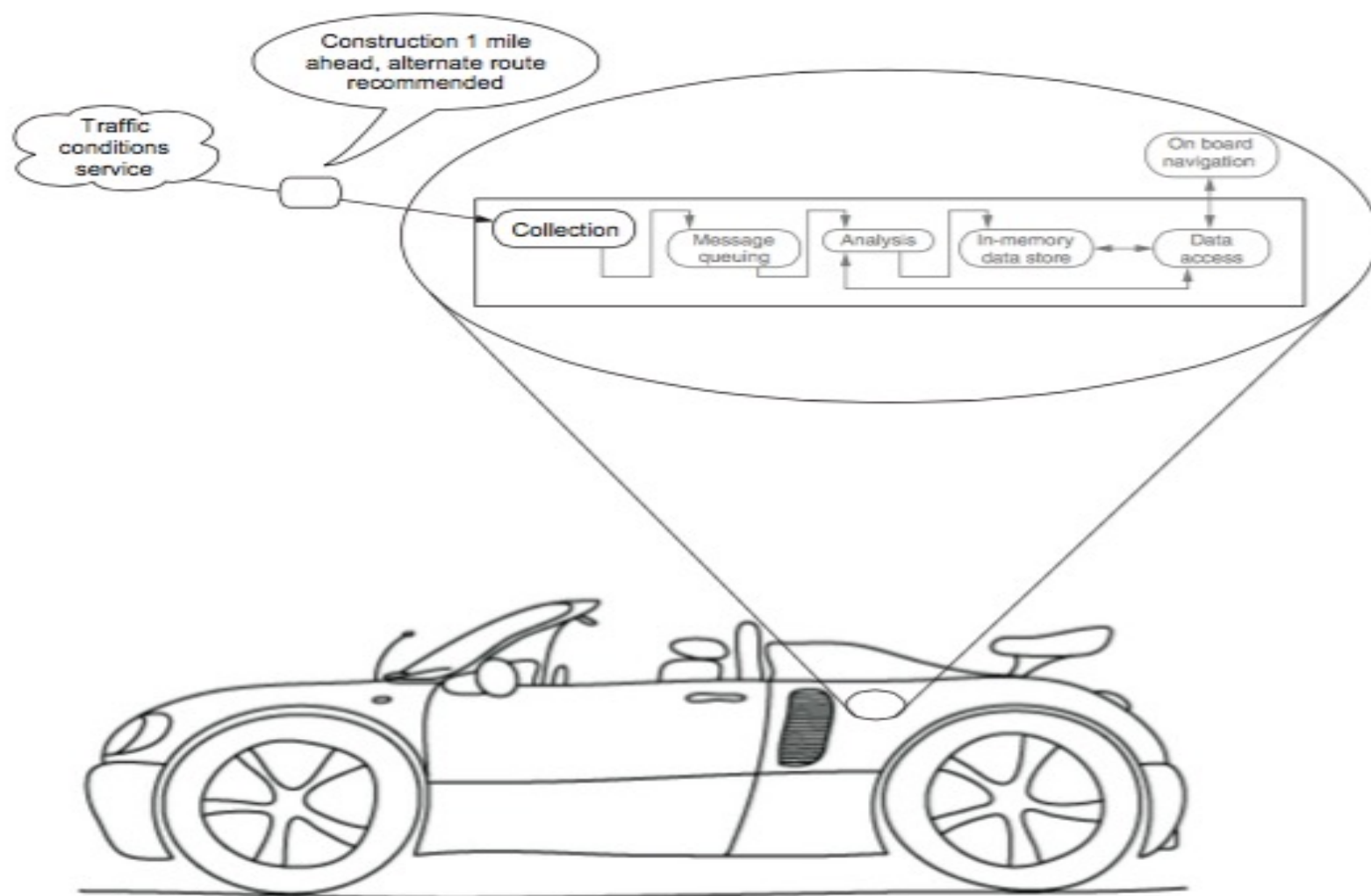
The request and the response happen over the same connection.



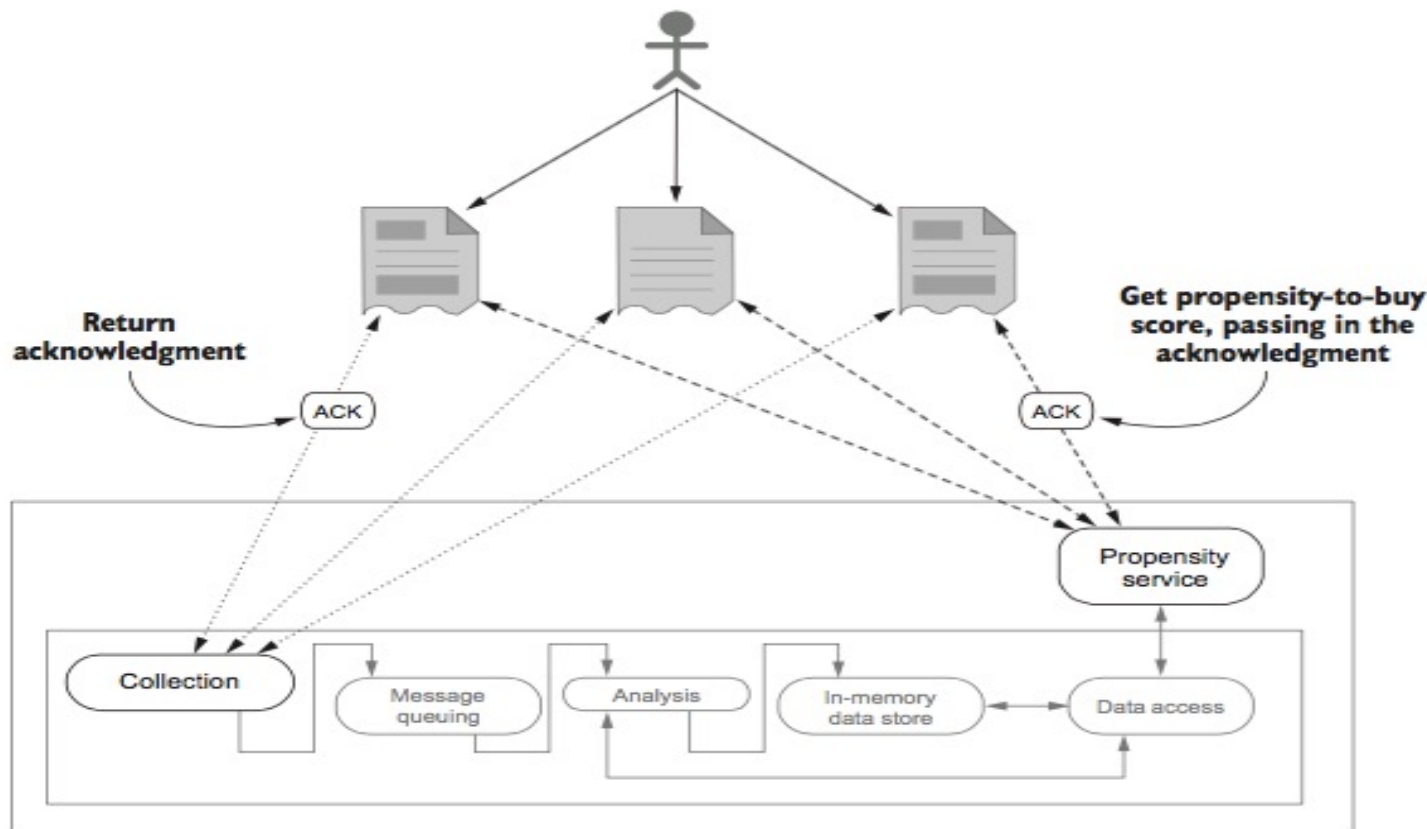
(b) Half-async RR



c) Full-async RR

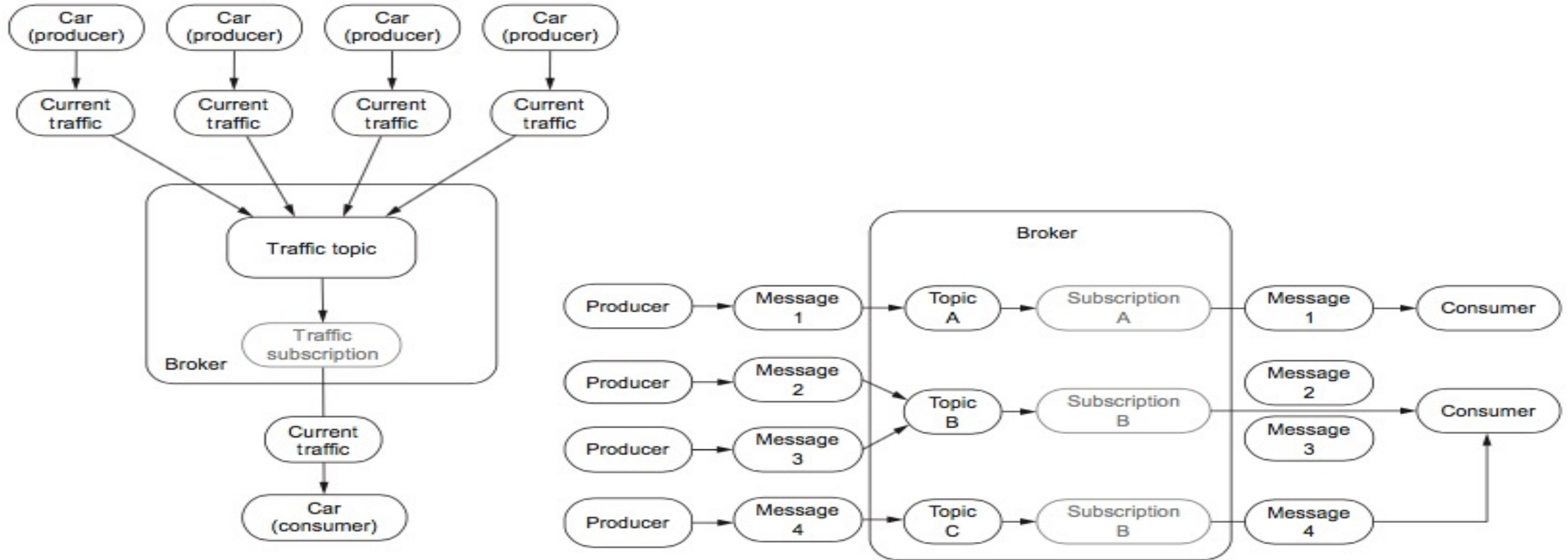


Request/acknowledge



- As the visitor is browsing our site, we're collecting data about each page they visit and every link they click
- The request/acknowledge pattern occurs on the first page they visit
- The acknowledgment is nothing more than a unique identifier
- When we call the propensity service, we can pass the unique identifier we obtained on the very first visit.

Publish/subscribe

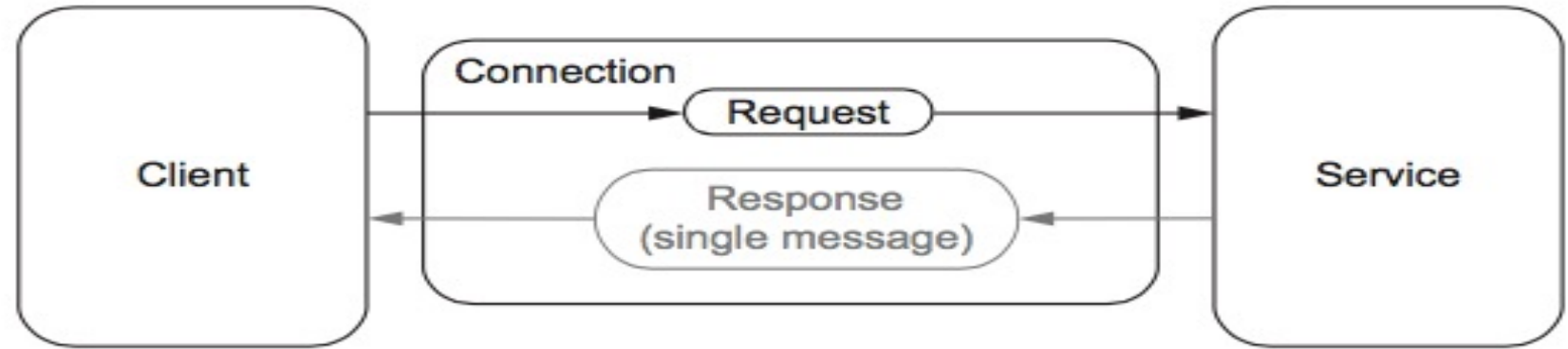


One-way

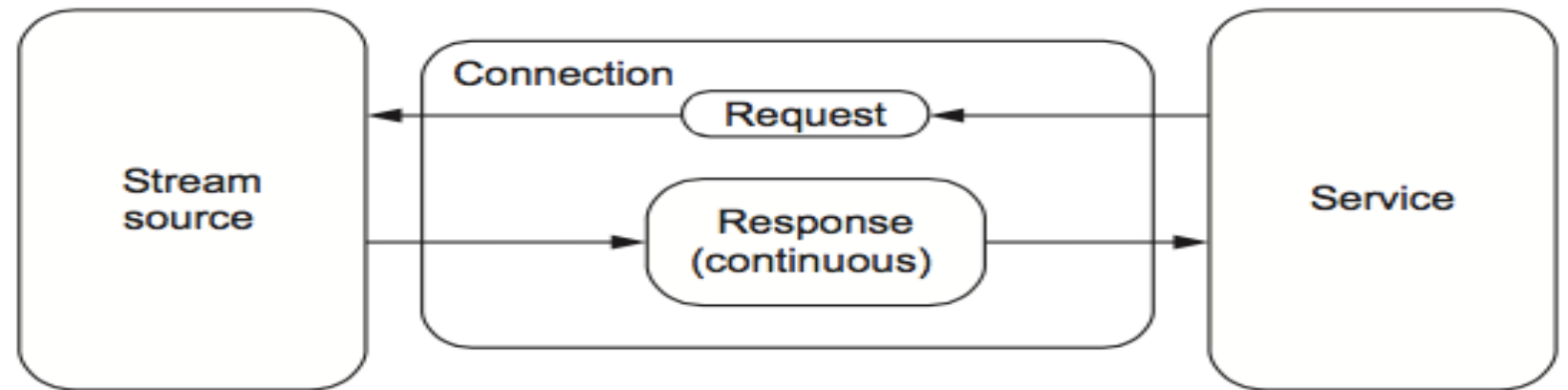
- The “fire and forget” message pattern
- The system making the request doesn’t need a response
- The client does not even know whether the request was received by the service
- Ex:
 - (Environment) sensors
 - Servers send data to the Monitoring System
 - RFID tag to RFID receiver

Stream

- The stream pattern flips things around as in other patterns
 - the service becomes the client
- A single request results in no data or a continual flow of data as a response
- The collection tier connects to a stream source and pulls data in

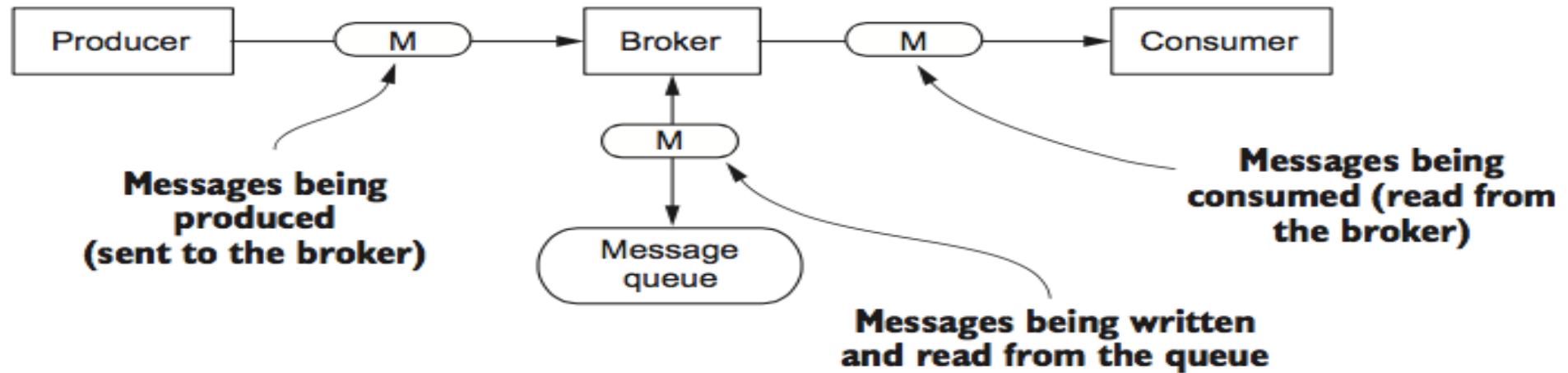


Request/response optional

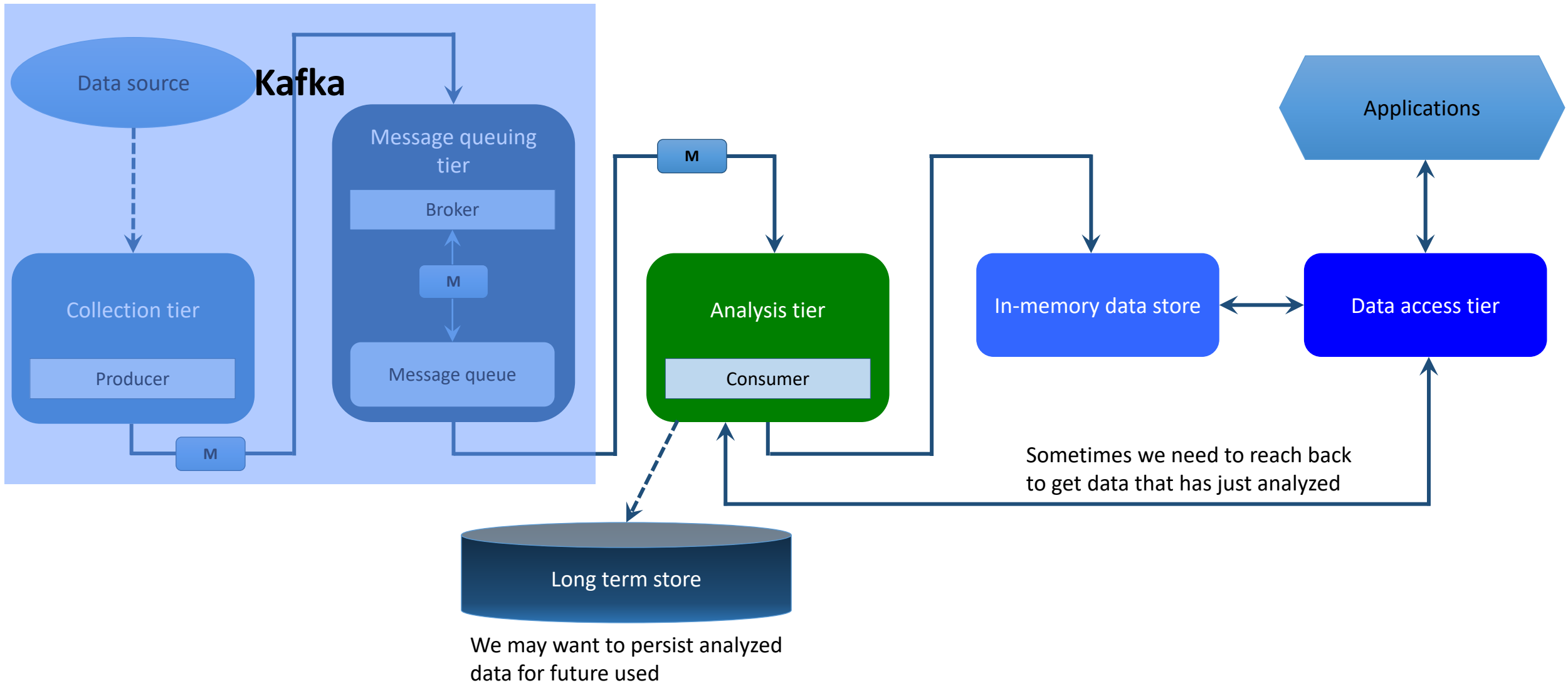


Streaming

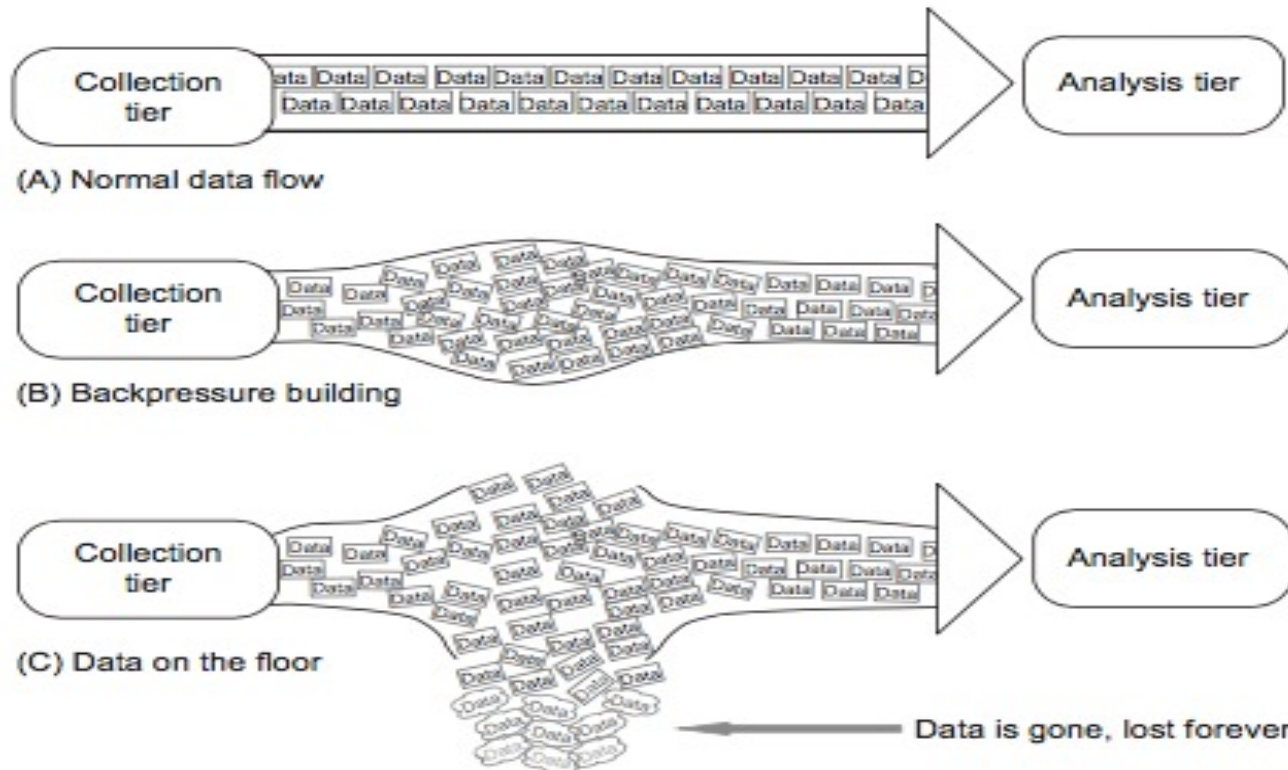
Producer, broker and consumer



- The three core parts to a message queuing system
 - **Producer** sends a message to a broker
 - **Broker** puts the message into a queue
 - **Consumer** reads the message from the broker



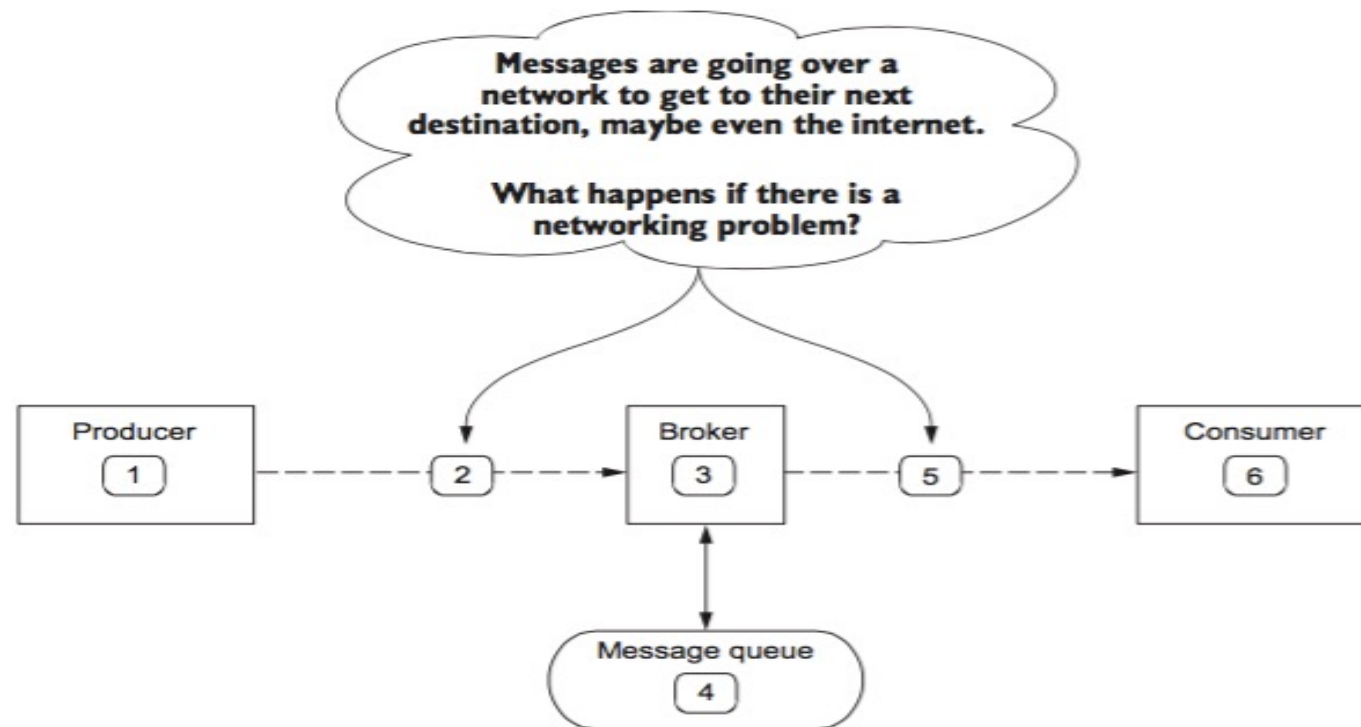
Isolating producers from consumers



- *Step A:* this looks pretty normal and is what we would like to see
- *Step B:* we can tell something is not quite right backpressure is building
- *Step C:* our data pipe broke under pressure, and data is now virtually dropping onto the floor and is gone forever.

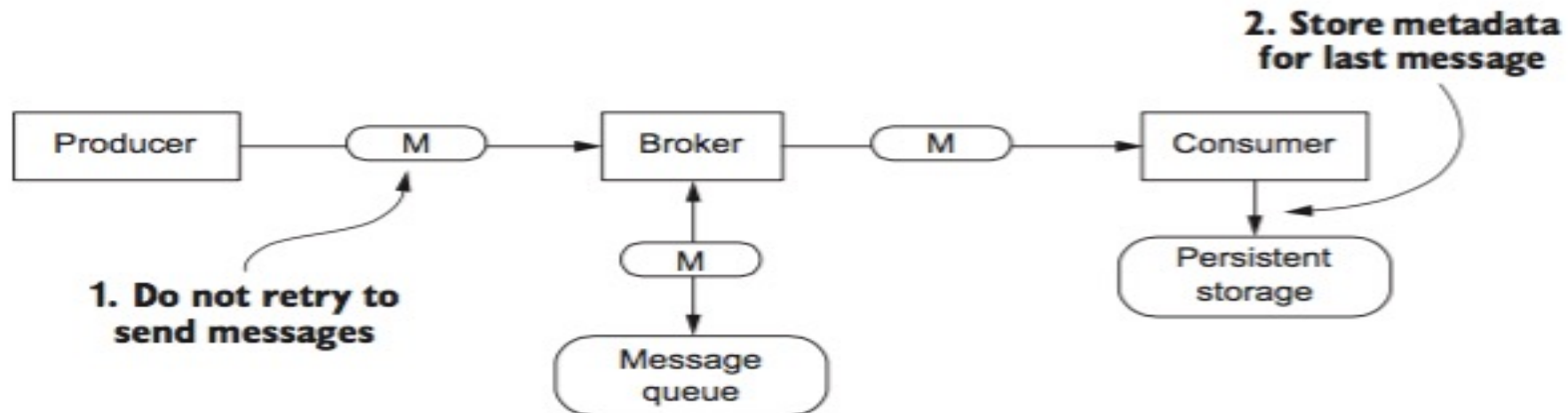
Message delivery semantics

- *At most once*: a message may get lost, but it will never be reread by a consumer
- *At least once*: a message will never be lost, but it may be reread by a consumer
- *Exactly-once*: a message is never lost and is read by a consumer once and only once.



Exactly-once semantics

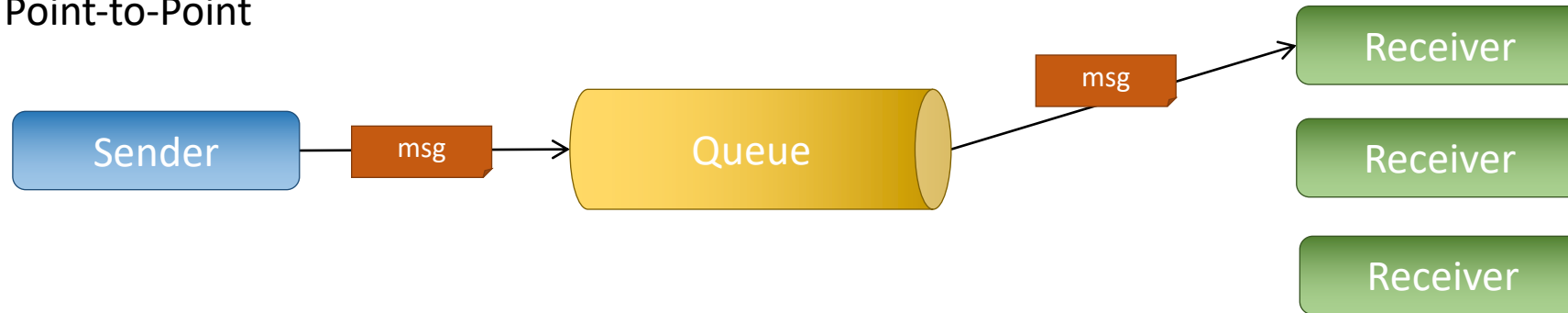
- Apache Kafka and Apache ActiveMQ: NOT support
- Enough metadata about the messages that you can implement exactly-once semantics with some coordination between producer(s) and consumer(s):
 - *Do not retry to send messages*
 - Read data from the broker to verify that the message you didn't receive an acknowledgment
 - *Store metadata for last message*
 - Storing some data about the last message we read, e.g. message offset in Apache Kafka
 - Taking into consideration is what to do if there's a failure storing the metadata



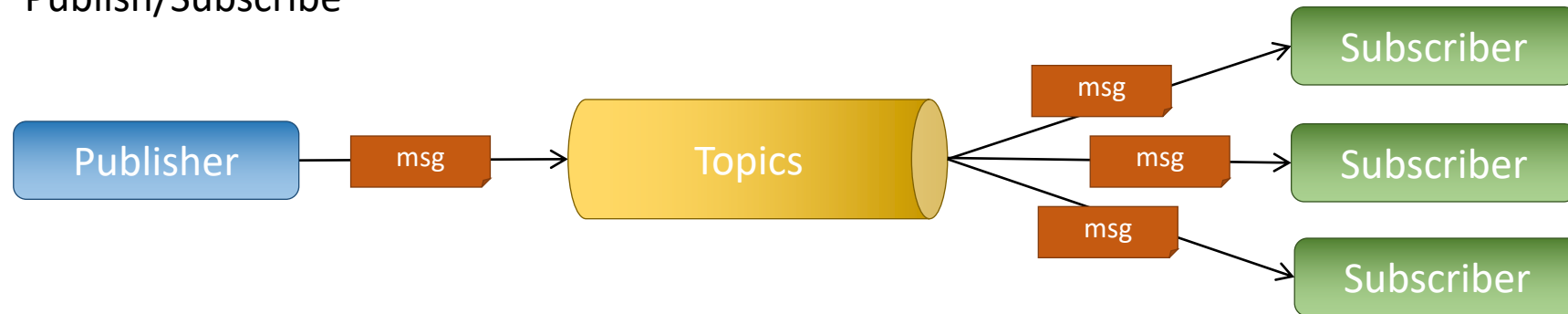
Kafka

Message models

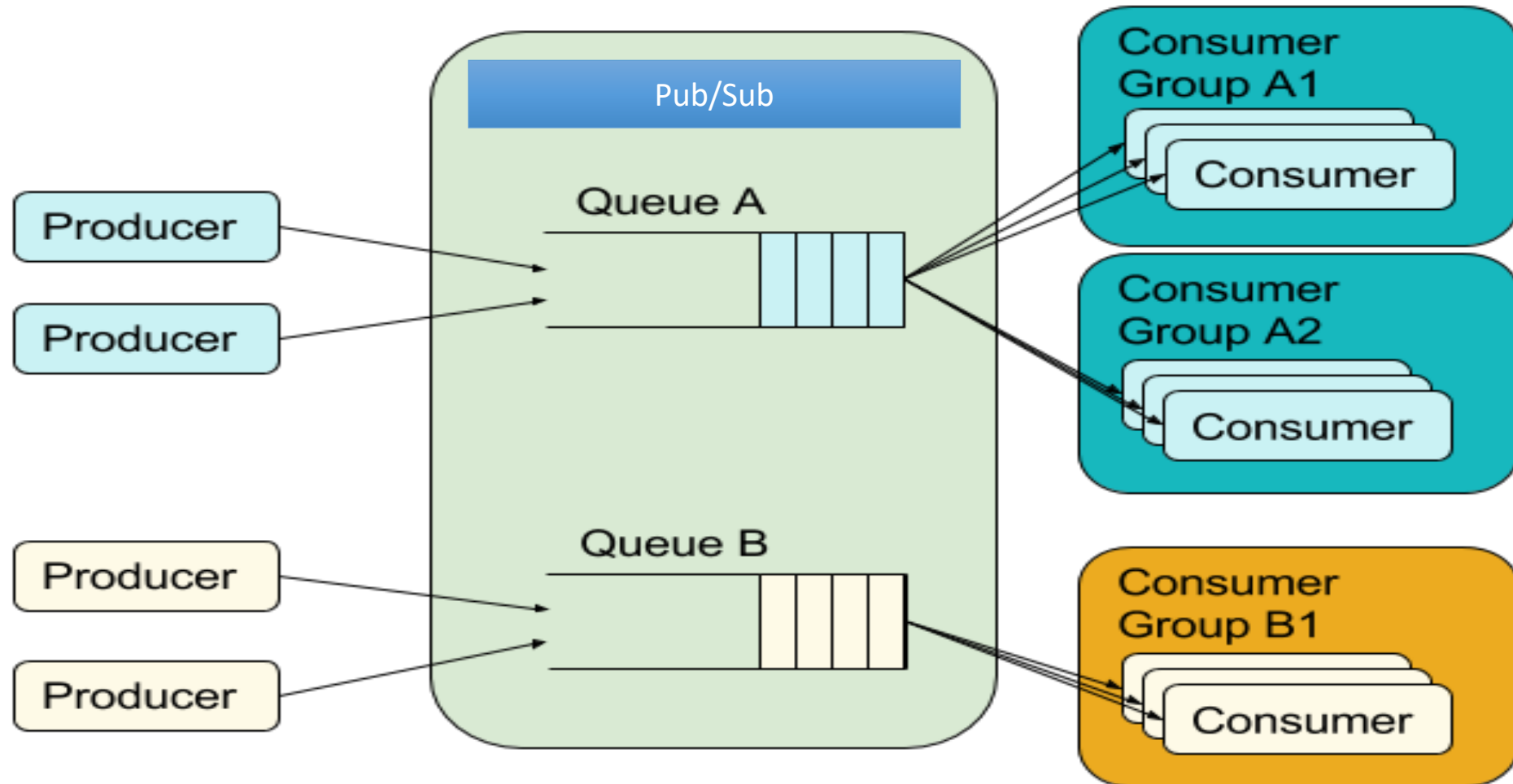
Point-to-Point



Publish/Subscribe



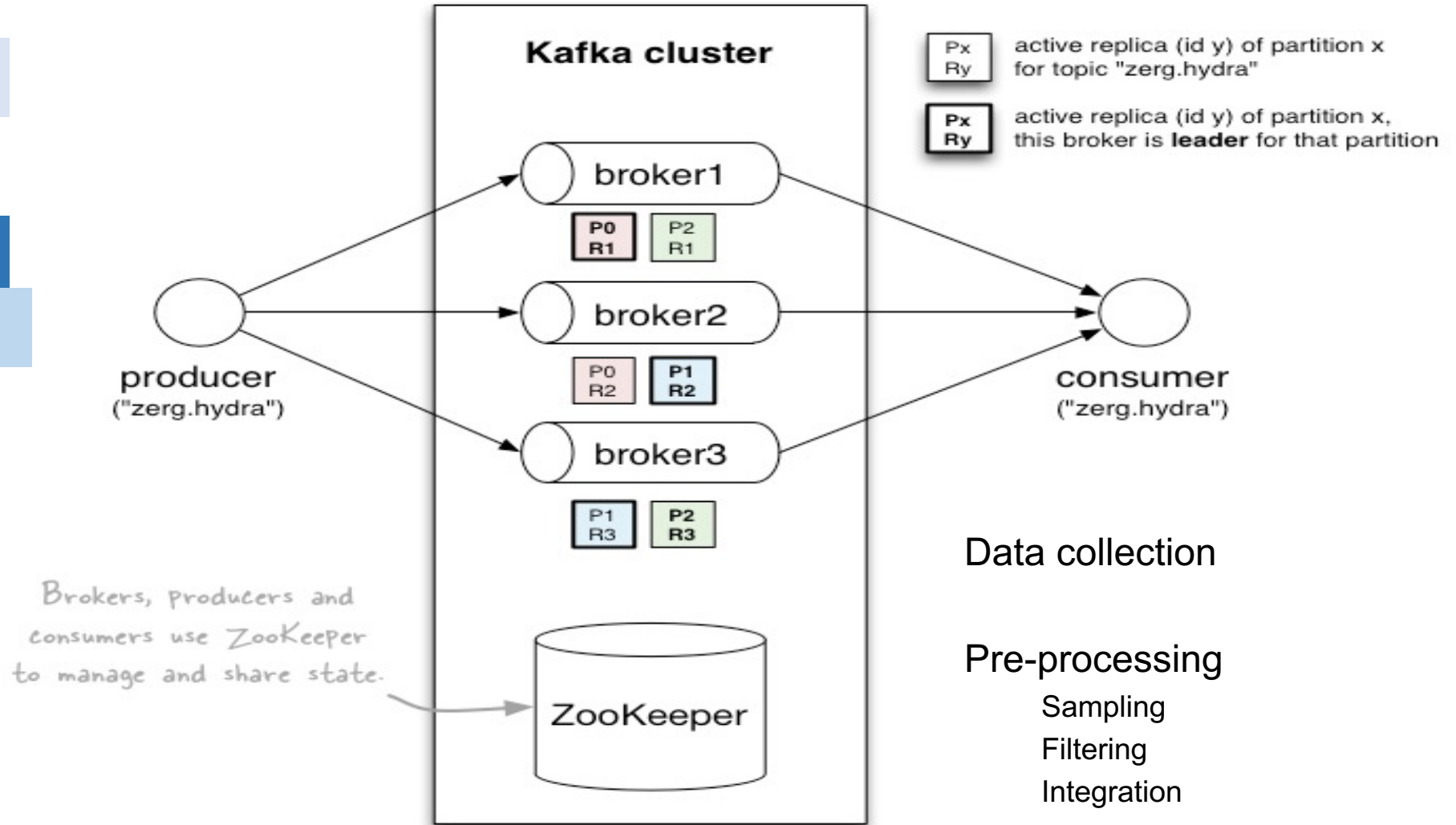
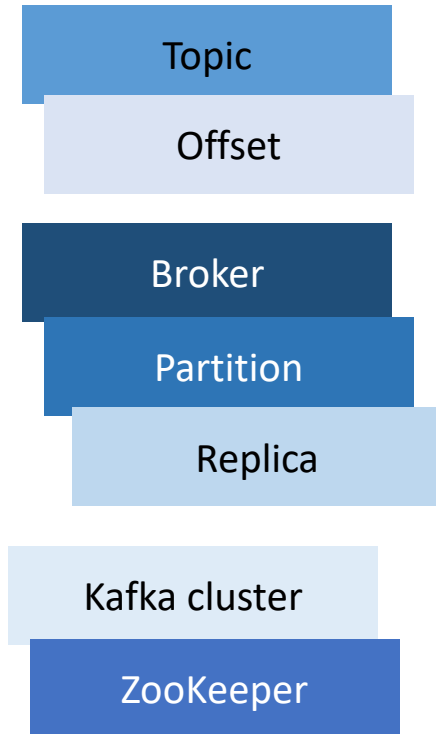
Publish/Subscribe model



Kafka

- Kafka is a “publish-subscribe messaging rethought as a distributed commit log”
- Fast
- Scalable
- Durable
- Distributed

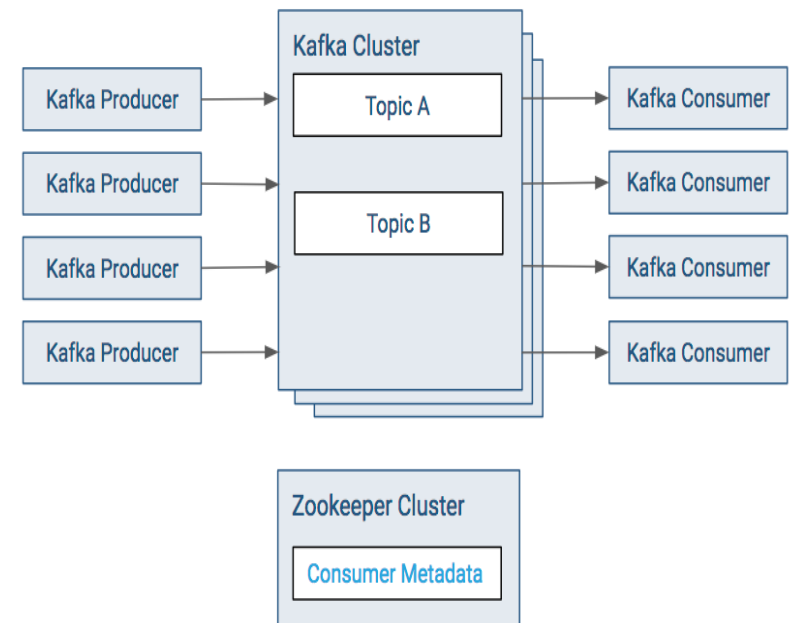
Apache Kafka



ZooKeeper

Zookeeper is a centralized service to maintain naming and configuration data, provide flexible and robust synchronization within distributed systems

- Zookeeper keeps track of status of the Kafka brokers, topics, partitions,... and notify all changes to Kafka
- **Kafka cannot run without Zookeeper!**



Semantics

There are 3 delivery semantics to commit the new offset:

Exactly one:

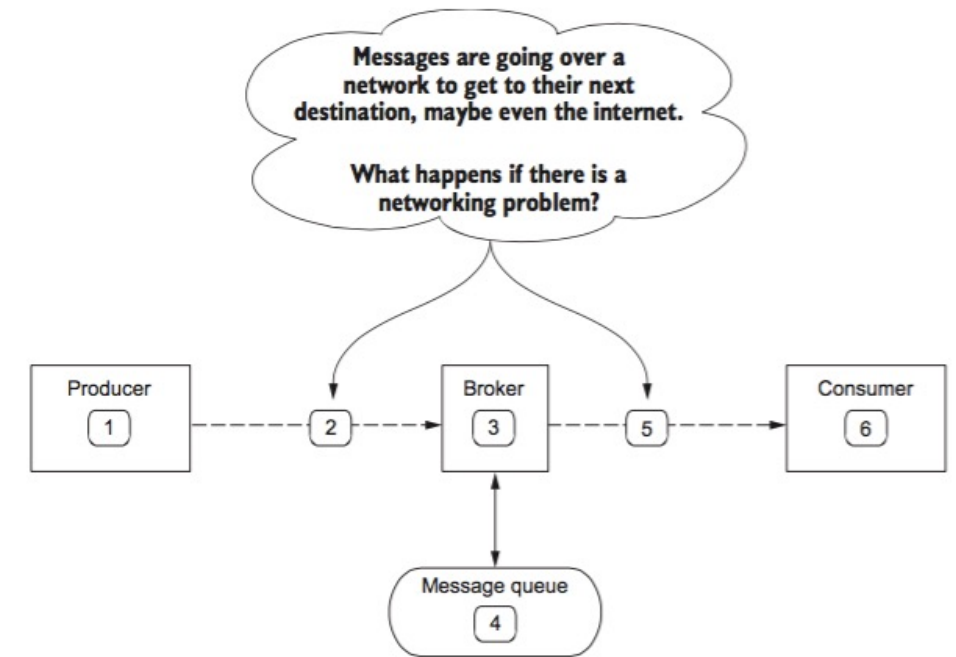
- Guarantees that all messages will always be delivered exactly once
- Only possible through the Streams API in Apache Kafka

At most once:

- Offsets are updated as soon as the message is **retrieved**
- Will **lose current message** if there are any exceptions while processing

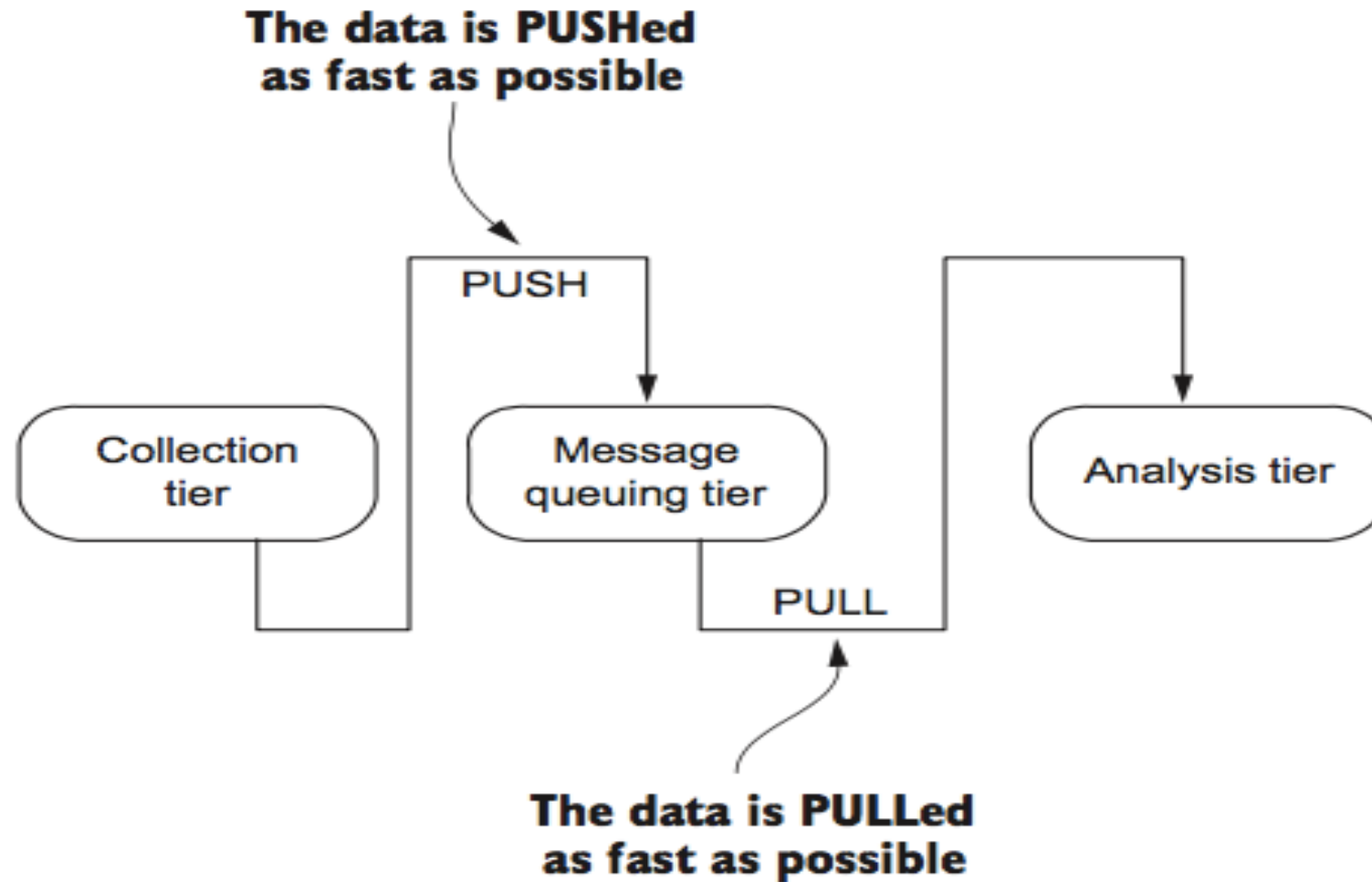
At least once (*preferred*):

- Offsets are updated after the message is **processed successfully**
- If there are any errors, the message will be **read again until operation successfully** -> can cause infinite loop
- A message can be processed twice or more -> Cause multiple effects (ex: duplicated notifications, emails)

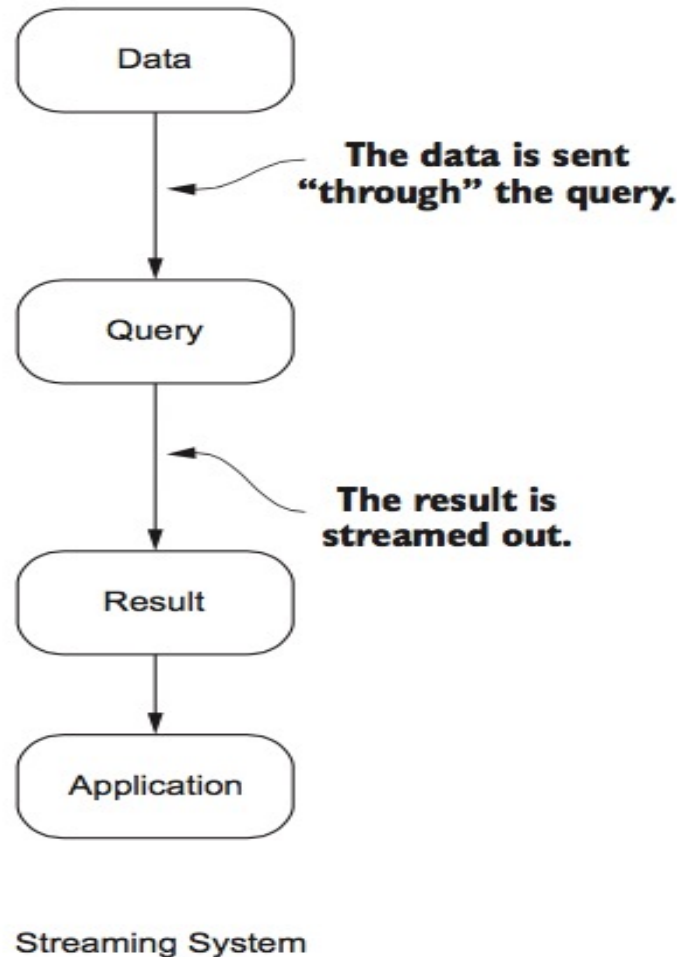
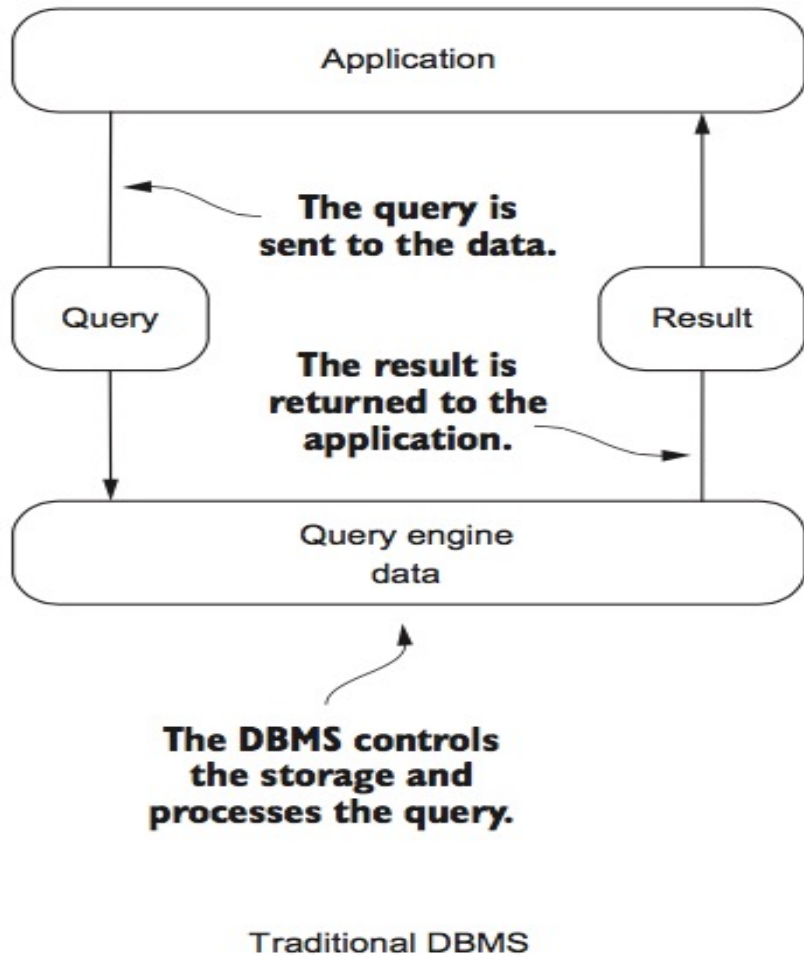


Analyzing streaming data

PUSH/PULL



Non-streaming & streaming system



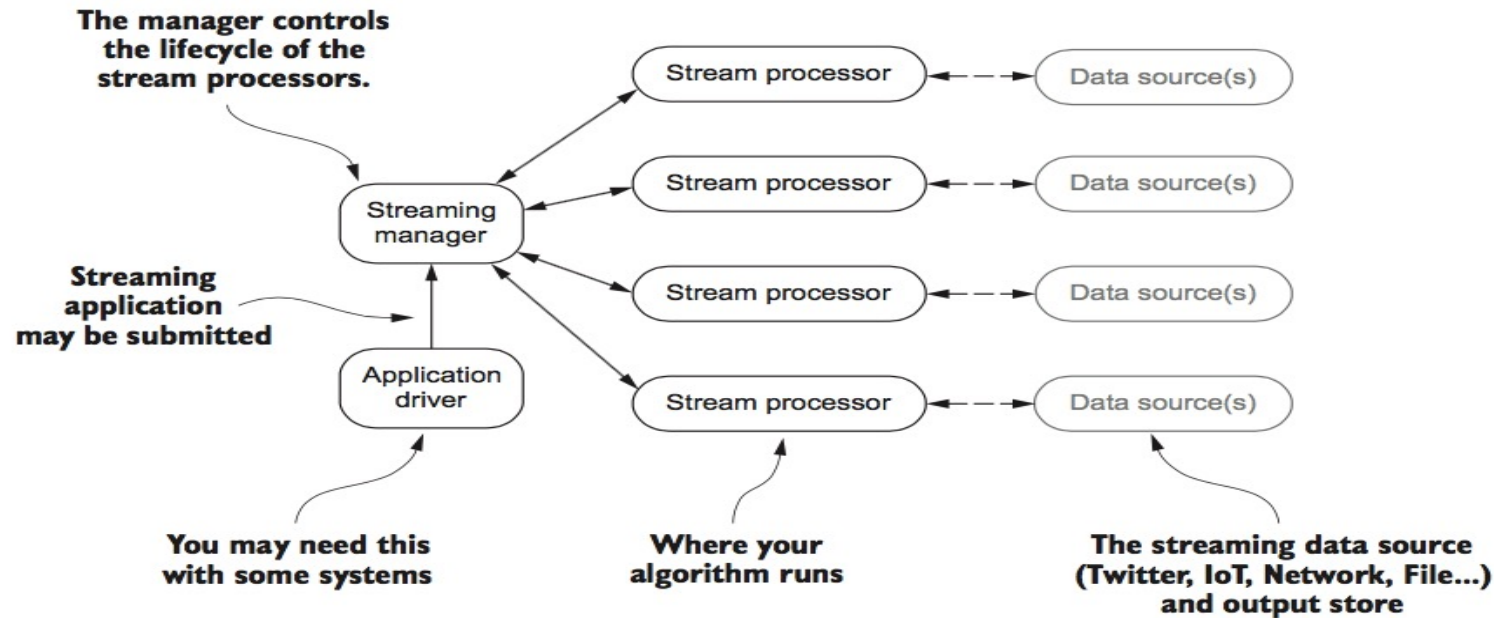
- A traditional DBMS (RDBMS, Hadoop, HBase, Cassandra, and so on):
 - In those non-streaming systems the data is at rest, and we query it for answers
- A streaming system:
 - *In-flight data*: The data is moved through the query
 - *Continuous query model*: the query is constantly being evaluated as new data arrive

Comparison of traditional DBMS to streaming system

	DBMS	Streaming system
Query model	Queries are based on a one-time model and a consistent state of the data. In a one-time model, the user executes a query and gets an answer, and the query is forgotten. This is a pull model.	The query is continuously executed based on the data that is flowing into the system. A user registers a query once, and the results are regularly pushed to the client.
Changing data	During down time, the data cannot change.	Many stream applications continue to generate data while the streaming analysis tier is down, possibly requiring a catch-up following a crash.
Query state	If the system crashes while a query is being executed, it is forgotten. It is the responsibility of the application (or user) to re-issue the query when the system comes back up.	Registered continuous queries may or may not need to continue where they left off. Many times it is as if they never stopped in the first place.

Distributed stream-processing architecture (I)

- Tools: (Apache) [Spark Streaming](#), [Storm](#), [Flink](#), and [Samza](#)
 - A component that your streaming application is submitted to; this is similar to how Hadoop Map Reduce works. Your application is sent to a node in the cluster that executes your application
 - Separate nodes in the cluster execute your streaming algorithms
 - Data sources are the input to the streaming algorithms

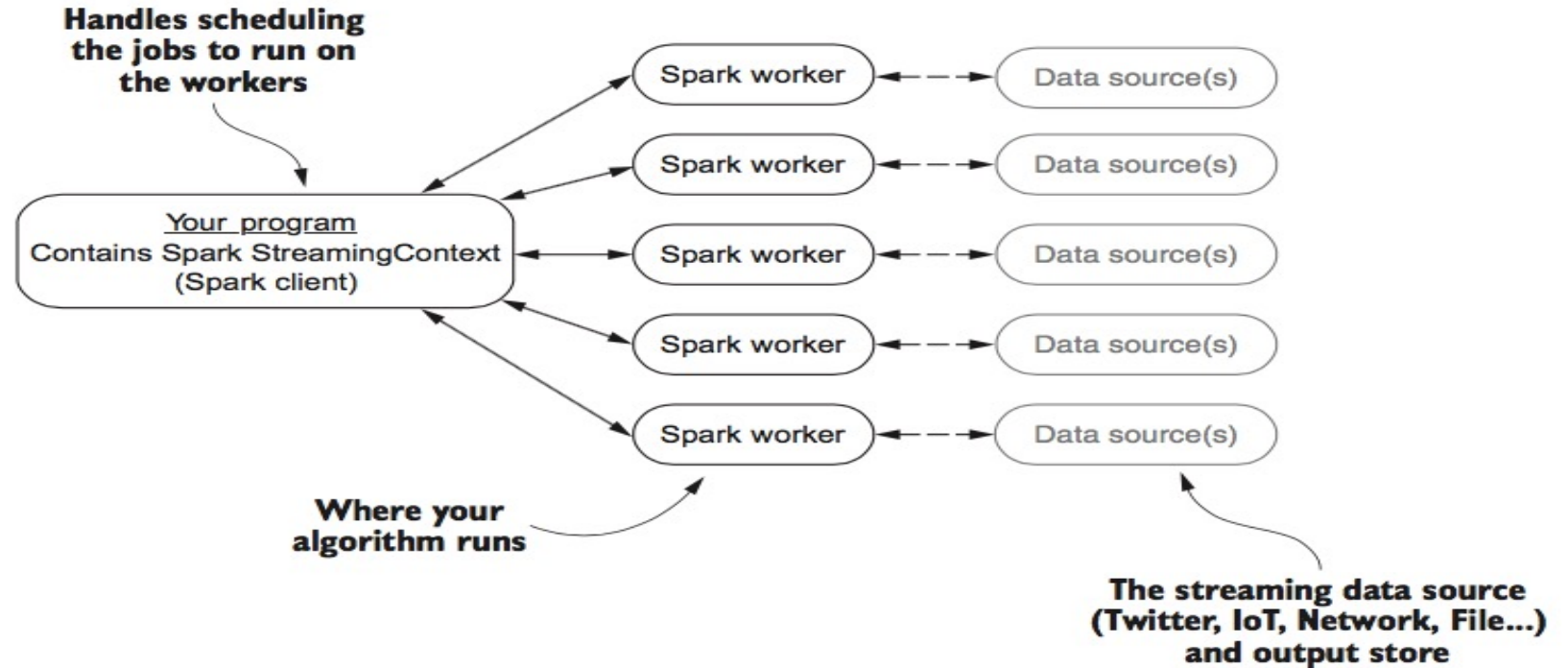


Distributed stream-processing architecture (2)

- *Application driver*: with some streaming systems, this will be the client code that defines your streaming programming and communicates with the streaming manager
- *Streaming manager*: the streaming manager has the general responsibility of getting your streaming job to the stream processor(s); in some cases it will control or request the resources required by the stream processors
- *Stream processor*: the place where your job runs; although this may take many shapes based on the streaming platform in use, the job remains the same: to execute the job that was submitted
- *Data source(s)*: This represents the input and potentially the output data from your streaming job. With some platforms your job may be able to ingest data from multiple sources in a single job, whereas others may only allow ingestion from a single source.

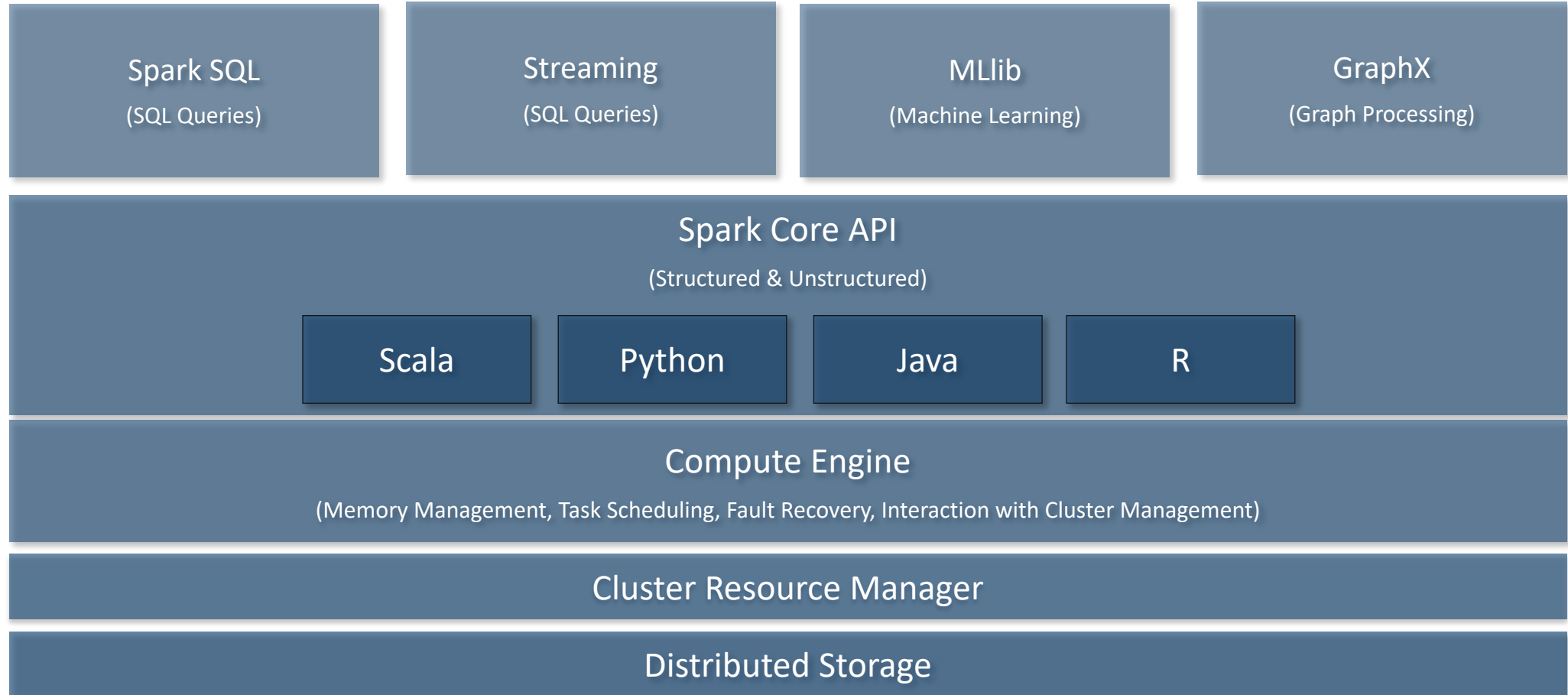
Apache Spark

- The de facto platform for general-purpose distributed computation
- Programming languages: Java, Scala, Python, and R
- Modules:
 - [Spark Streaming](#)
 - MLlib (machine learning)
 - SparkR (integration with R)
 - GraphX (for graph processing)



- [Spark StreamingContext](#) is the driver
- [A job in Spark Streaming](#) is the logic of your program that's bundled and passed to the Spark workers
- [Spark workers](#): which run on any number of computers (from one to thousands) and are where your job (your streaming algorithm) is executed. They receive data from an external data source and communicate with the Spark StreamingContext that's running as part of the driver.

Apache Spark

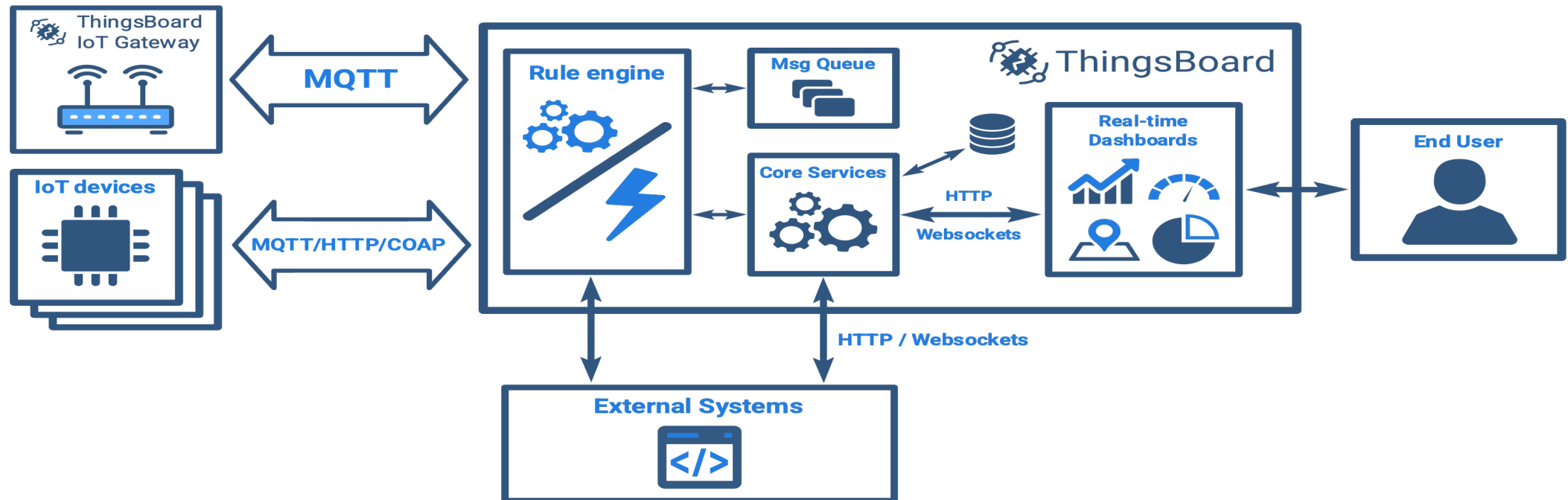


Data analytics

Message delivery semantics

- *At-most-once* - a message may get lost, but it will never be processed a second time => **Simple**
- *At-least-once* - a message will never be lost, but it may be processed more than once
 - If every time the streaming job receives the same message, it produces the same result
=> the duplicate-messages situation
- *Exactly-once* - a message is never lost and will be processed only once
 - Detect and ignore duplicates

ThingsBoard



- Collection data node
- Support many protocols
- Rule engine
 - Pre-processing: sampling, filtering, integration
- Message queue + etc.

=> A small scale solution (do not need Kafka + Spark)

Algorithms for (streaming) data analytics

Streaming data queries

- *Ad-hoc queries* - These are queries asked one time about a stream
 - Ex: What is the maximum value seen so far in the stream? This style of query is the same kind you would execute against an RDBMS
- *Continuous queries*: These are queries that are, in essence, asked about the stream at all times
 - Ex: Determine the maximum value ever seen in the stream emitted every five minutes and generate an alert if it exceeds a given threshold

Product	Query language support
Apache Storm	As of version 1.1.0 Apache Storm has had SQL support (http://storm.apache.org/releases/1.1.0/storm-sql.html). As of this writing it is still considered experimental and not ready for production use
Apache Samza	Since version 0.9 of Apache Samza there has been a JIRA open for adding query language support. As of this writing, that JIRA is still open, and Samza does not have any query language support: https://issues.apache.org/jira/browse/SAMZA-390
Apache Flink	Table API supporting SQL-like expressions (http://ci.apache.org/projects/flink/flink-docs-release-0.9/libs/table.html)
Apache Spark Streaming	SparkSQL/Hive language support (http://spark.apache.org/docs/latest/sql-programming-guide.html)

Constrains and relaxing

- *One-pass*

- We must assume that the data is not being archived and that we only have one chance to process it
- Many traditional data-mining algorithms are iterative and require multiple passes over the data

- *Concept drift*

- This is a phenomenon that may impact your predictive models. Concept drift may happen over time as your data evolves and various statistical properties of it change

- *Resource constraints*

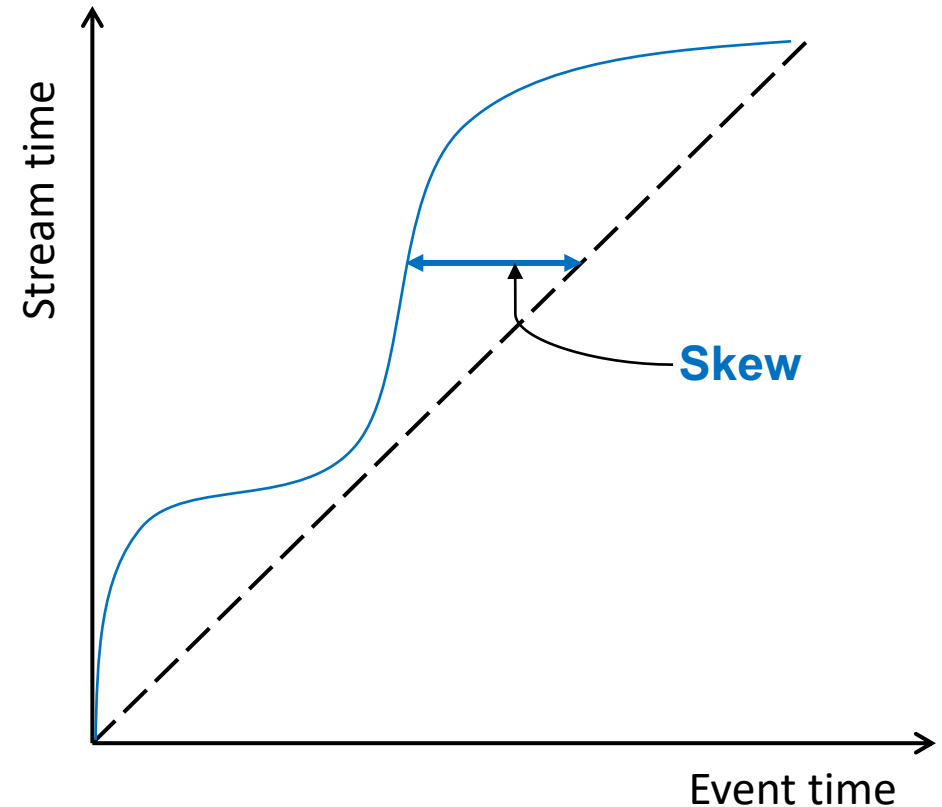
- A temporary peak in the data speed or volume => an algorithm may have to drop tuples that can't be processed in time, called *load shedding*

- *Domain constraints*

- huge collected data => challenges in analytics.

Stream time and event time

- *Stream time* is the time at which an event enters the streaming system: $T_{\text{stream}}(e)$
- *Event time* is the time at which the event occurs: $T_{\text{event}}(e)$
- $T_{\text{stream}}(e) > T_{\text{event}}(e)$
- *Time skew* = $T_{\text{stream}}(e) - T_{\text{event}}(e)$

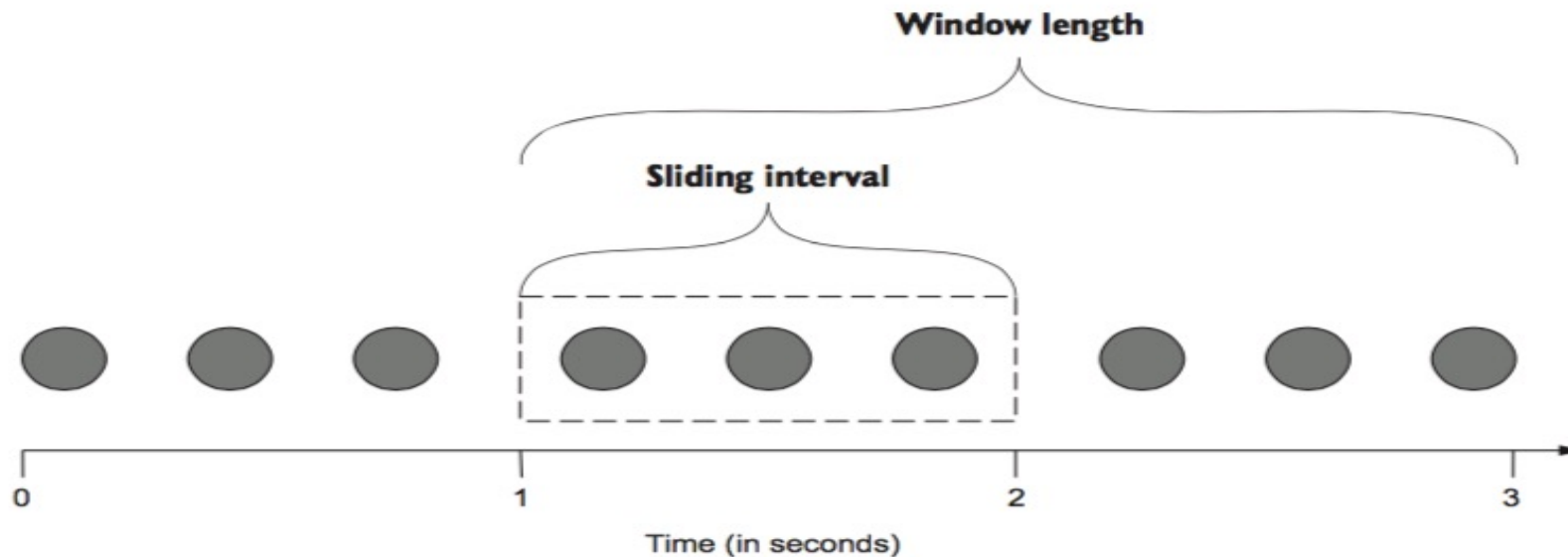


Windows of time

- Due to its size and never-ending nature, the stream processing engine can't keep an entire stream of data in memory
 - Cannot perform traditional batch processing on it
- A *window of data* represents a certain amount of data that we can perform computations on
 - The *trigger policy* defines the rules a stream-processing system uses to notify our code that it's time to process all the data that is in the window
 - The *eviction policy* defines the rules used to decide if a data element should be evicted from the window
 - Both policies are driven by either time or the quantity of data in the window.

Sliding window

- The *sliding* window technique uses **eviction** and **trigger policies** that are based on time
- The **window length** represents the eviction policy—the duration of time that data is retained and available for processing
- The **sliding interval** defines the trigger policy

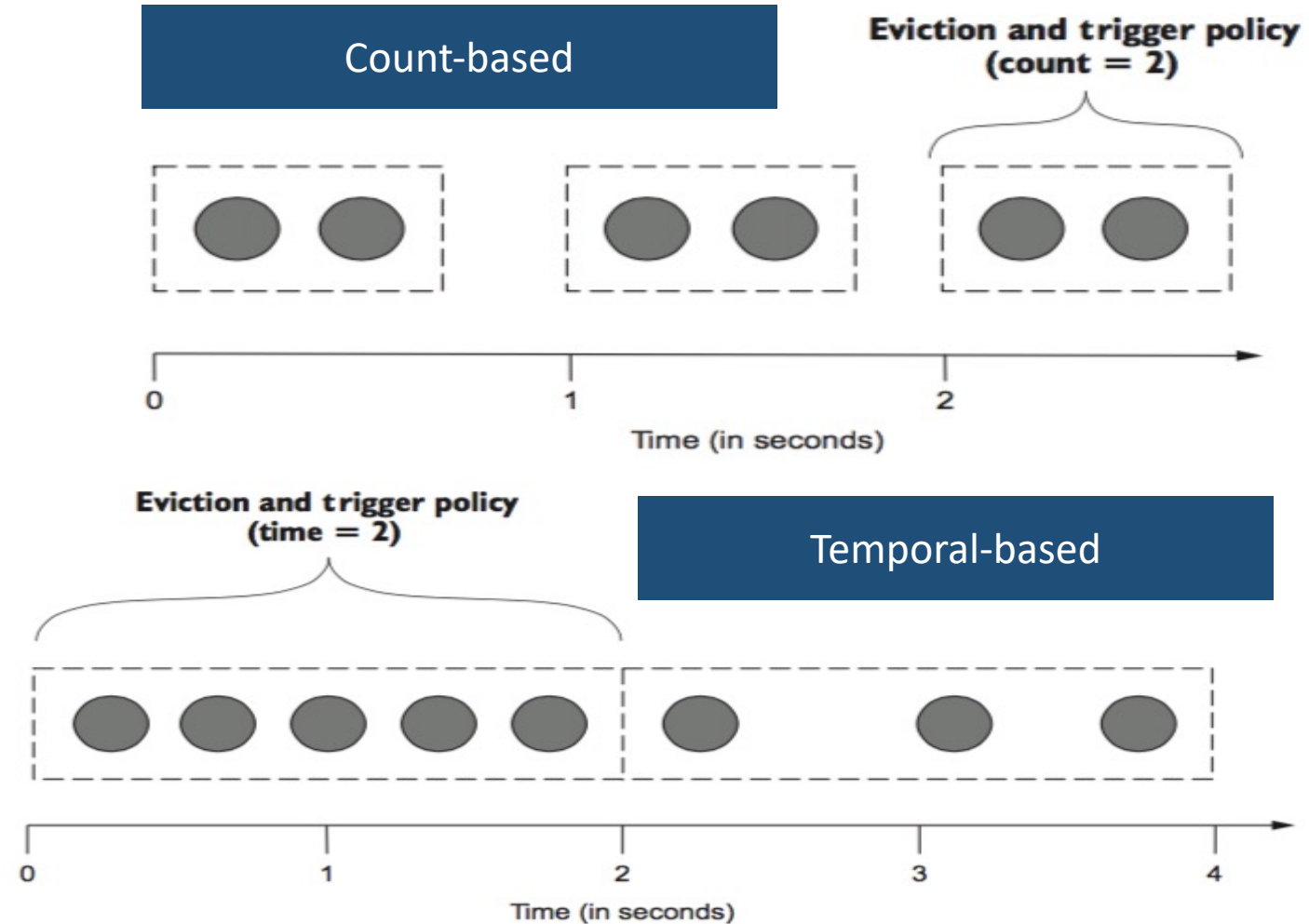


Sliding window support in popular stream-processing frameworks

Framework	Sliding window	Event or stream time	Comments
Spark Streaming	Yes	Stream time	Spark Streaming doesn't allow custom policies.
Storm	No	N/A	Storm doesn't provide native support for sliding windowing, but it could be implemented using timers.
Flink	Yes	Both	Flink allows a user to define a custom policy and trigger policies.
Samza	No	N/A	Samza doesn't provide direct support for sliding windows.

Tumbling window

- The eviction policy is always based on the window being full
- The trigger policy is based on either the count of items in the window or time
 - Count-based
 - Temporal-based



Tumbling window support in popular stream-processing frameworks

Framework	Count	Temporal	Comments
Spark Streaming	No	No	Currently you would need to build this.
Storm	Yes	Yes	Although Storm does not have the native windowing support, we can easily implement this.
Flink	Yes	Yes	Flink has built-in support for both types of tumbling windows.
Samza	No	Yes	Samza does not provide direct support for sliding windows.

Algorithms

- Sampling

- Since we cannot store the entire stream, one obvious approach is to store a sample

- Membership

- Has this stream element ever occurred in the stream before?

- Frequency

- How many times has stream element X occurred?

- Counting distinct elements

- Count the distinct items in a stream, but remember we are constrained by memory and don't have the luxury of storing the entire stream

Sampling

Distributed (big data) Sampling

■ Fixed-proportion

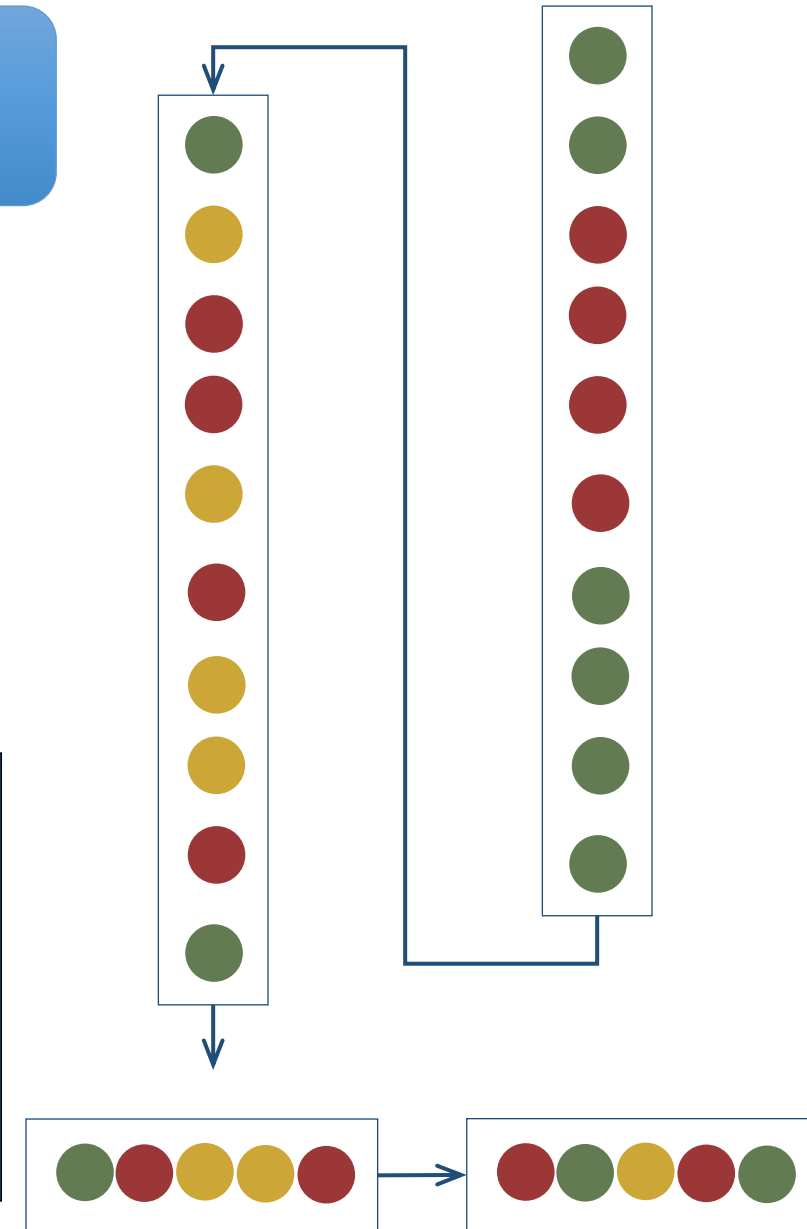
- Sample a fixed proportion of elements in the stream (say 1 in 10)
- Use a hash function that hashes keys of tuples uniformly into 10 buckets

■ Fixed-size

- Maintain a random sample of fixed size over a potentially infinite stream
- Reservoir sampling

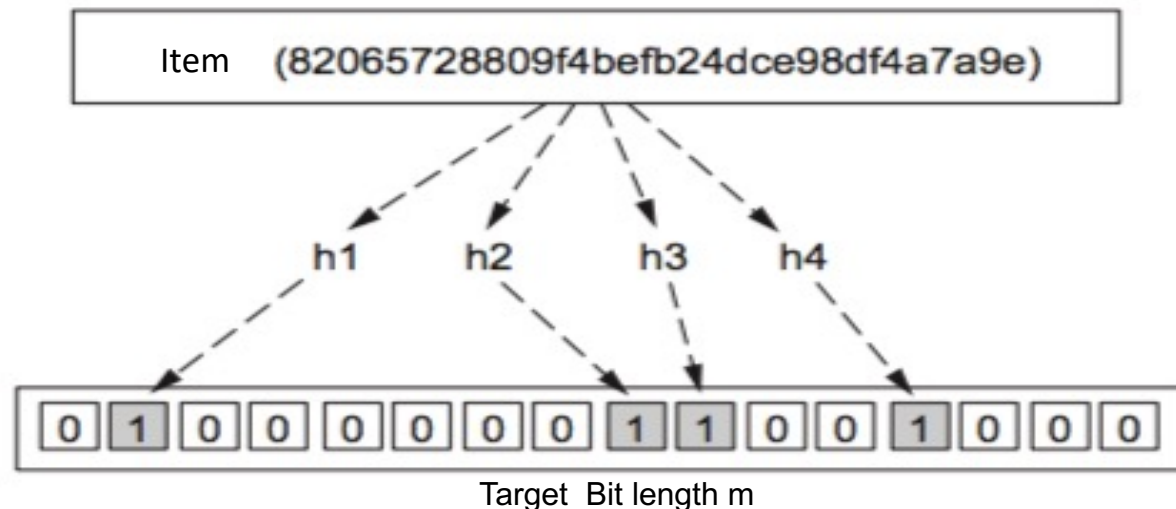
Reservoir sampling

- Store all the first s elements of the stream to S
- Suppose we have seen $n-1$ elements, and now the n^{th} element arrives ($n > s$)
 - With probability s/n , keep the n^{th} element, else discard it
 - If we picked the n^{th} element, then it replaces one of the s elements in the sample S , picked uniformly at random



Membership

- Has this stream element ever occurred in the stream before?
- Bloom filtering
 - a binary bit array of length m (Target_Bit) and is associated with a set of k independent hash functions
 - all item x and with all hash functions: $\text{bit}[h_i(x)] = 1$
 - *MEMBERSHIP* of stream element $z = \text{AND}(h_1(z), h_2(z), \dots, h_k(z))$



Spam mail with Bloom filtering (I)

- $y = 1\text{B e-mails/darts}$
- $x = 8\text{B targets/bits}$
- Probability that a given target won't be hit by any darts (i.e. zero bit)?
- Probability that given dart will not hit a given targets is $\frac{x-1}{x}$
- Probability that none of y darts will hit a given target is $\left(\frac{x-1}{x}\right)^y = \left(1 - \frac{1}{x}\right)^{x\frac{y}{x}} \rightarrow e^{-\frac{y}{x}}$
- Probability that any given bit will be zero is $e^{-\frac{y}{x}} = e^{-\frac{1}{8}}$
- Probability that given bit will be 1 is $1 - e^{-\frac{1}{8}} = 0.1175$
- Slightly less than $1/8=0.125$

Spam mail with Bloom filtering (2)

- S has m members, array has n bits and there are k hash functions
 - Targets $x=n$
 - Number of darts $y=km$
- We want proportion of 0 be large
 - So non-S will hash to zero at least once
 - Choose k to be n/m or less
 - Probability of 0 is $e^{-\frac{y}{x}} = e^{-\frac{km}{n}} = 0.37 = 37\%$
 - Probability of 1 is $1 - e^{-\frac{km}{n}}$
 - Probability of false positive: $(1 - e^{-\frac{km}{n}})^k$

Spam mail with Bloom filtering (3)

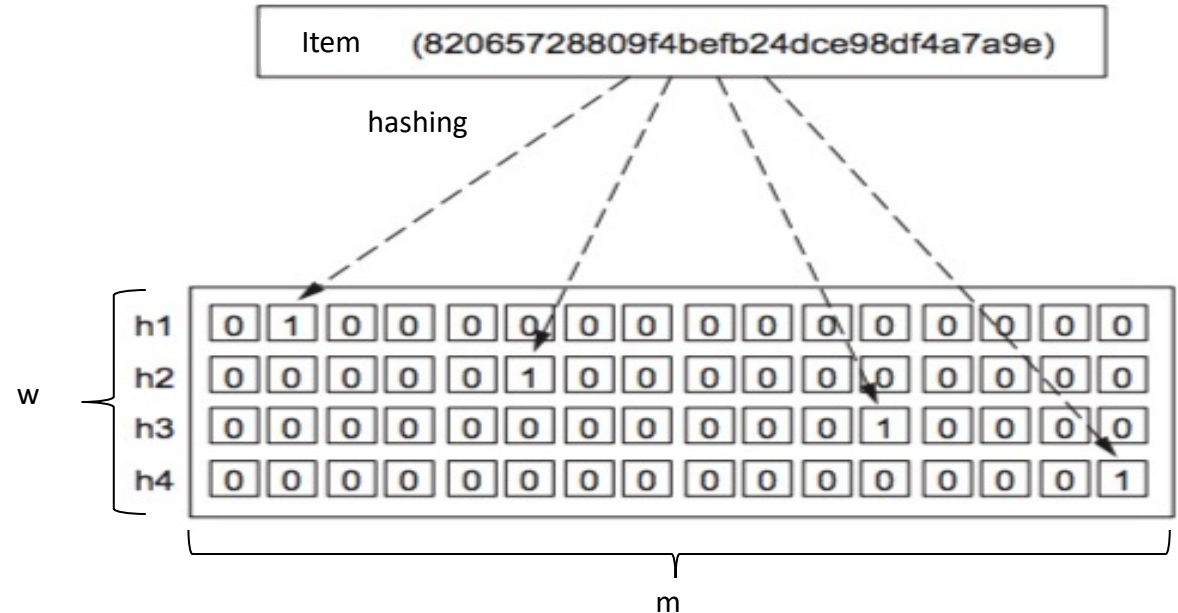
- In the previous example
 - Fraction on 1's is 0.1175
 - Also the probability of false positive
- Use two different hash functions
 - 2B darts on 8B targets
 - Probability of zero is $e^{-\frac{1}{4}}$
 - False positive: $(1 - e^{-\frac{1}{4}})^2 = 0.0493$

Frequency

- How many times has stream element X occurred?
- *Count-Min Sketch*
 - *A point query*: a particular stream element
 - *A range query*: frequencies in a given range.
 - *An inner product query*: the join size of two sketches; Ex: we may use this to provide a summarization to this question: What products were viewed after an ad was served?

Count-Min sketch

- w numeric arrays, often called *counters*, the length of each is defined by the length m
($m \ll n$ items)
- Each array is indexed starting at 0 and has a range of $\{0 \dots m - 1\}$
- Each counter must be associated with a different hash function ($h_1, h_2, h_3, \dots$), which must be pairwise independent
- Count first and compute the minimum next :
 - **Count step**: hash the item value using the hash function for each respective row and then increment the count for the cell the value hashes to by 1
 - **Min step**: The minimum value from the w cells represents the approximate count for the number of times the item was viewed
- This algorithm will never undercount, but could overcount
 - A width of 8 and a count of 128 (a 2-dimensional array of 8×128) the relative error was approximately 1.5%, and the probability of the relative error being 1.5% is 99.6%



Counting distinct elements

- Count the distinct items in a stream, but remember we are constrained by memory and do not have the luxury of storing the entire stream
- *Bit-pattern-based*
 - Observation of patterns of bits that occur at the beginning of the binary value of each element of the stream. Using the bit pattern - more specifically, the leading zeros in the binary representation of a hash of the stream element - the cardinality is determined
 - LogLog, HyperLogLog, and HyperLogLog++
- *Order statistics-based*
 - The algorithms in this class are based on order statistics, such as the smallest values that appears in a stream
 - MinCount and Bar-Yossef

Flajolet-Martin algorithm

- $h(a)$ is hashed to a bit string
- Number of zeros at the end is tail lengths of $h(a)$
- Let r be maximum tail length seen so far
- Estimate number of different elements as 2^r

The Flajolet-Martin algorithm

- Hash element to a sufficiently long bit string
 - More possible hash values than elements
 - 64 bits for URLs (264 values)
- Whenever number of distinct elements increases
 - Number of different hash-values increases
 - The probability of “special/unusual” hash-value increases
- Unusual value
 - Ending in many 0's

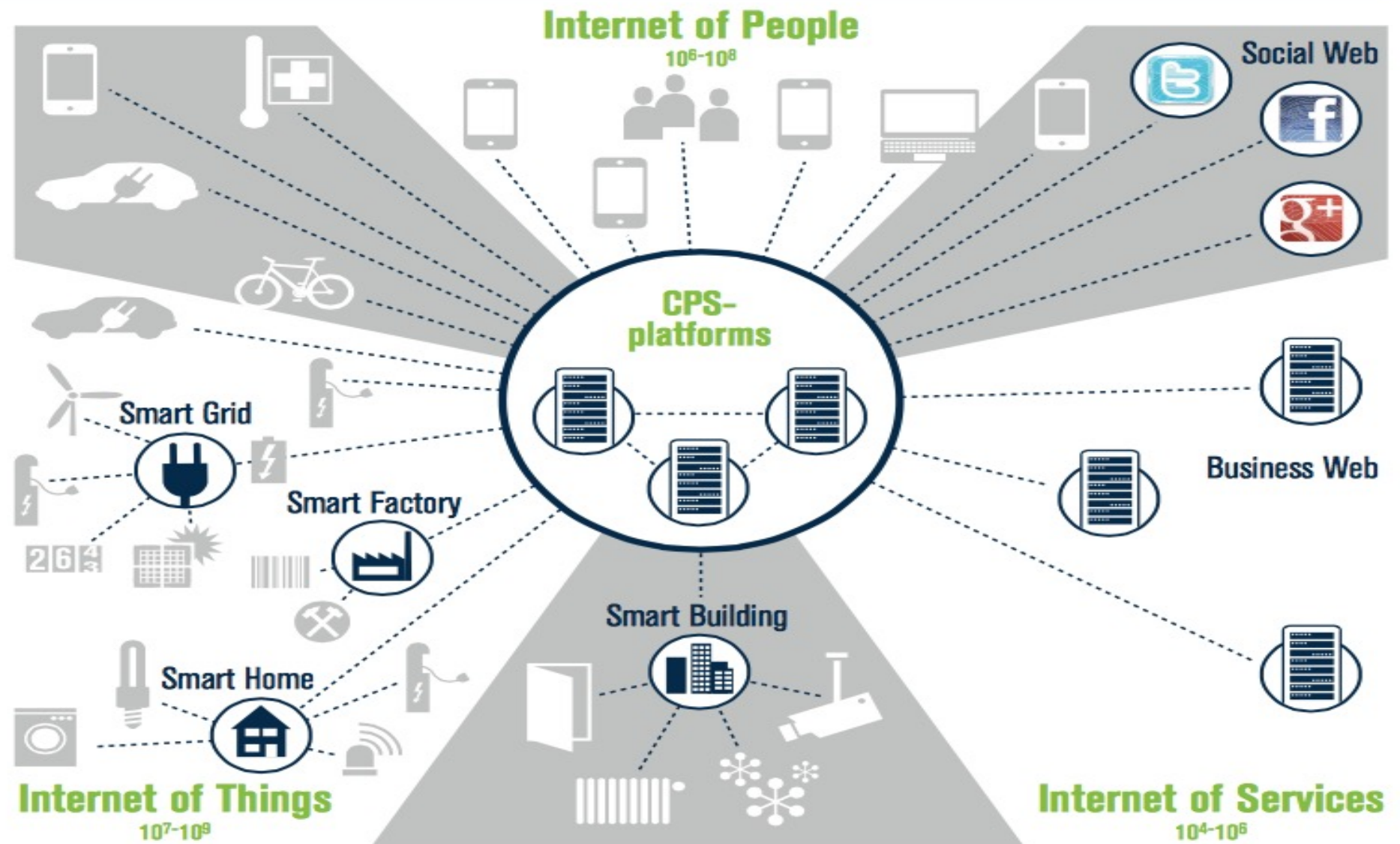
- Probability that any given element has tail length at least r is 2^{-r}
- For m distinct element in the stream, the probability that none of them has at least r is

$$(1 - 2^{-r})^m = (1 - 2^{-r})^{2^{-r}m2^r} \approx e^{-m2^{-r}}$$

- Estimate of m
 - For $m \gg 2^r$, $e^{-m2^{-r}}$ is small, at least one has r zeroes
 - For $m \ll 2^r$, $e^{-m2^{-r}}$ is large, no elements have r zeroes
 - Estimate the cardinality of M as $2^r/\Phi$, where $\Phi \approx 0.77351$.

Edge & Fog computing

Figure 4:
The Internet of Things and
Services – Networking
people, objects and systems



Source: Bosch Software Innovations 2012

INDUSTRIAL IoT DATA PROCESSING LAYER STACK

CLOUD LAYER

Big Data Processing
Business Logic
Data Warehousing

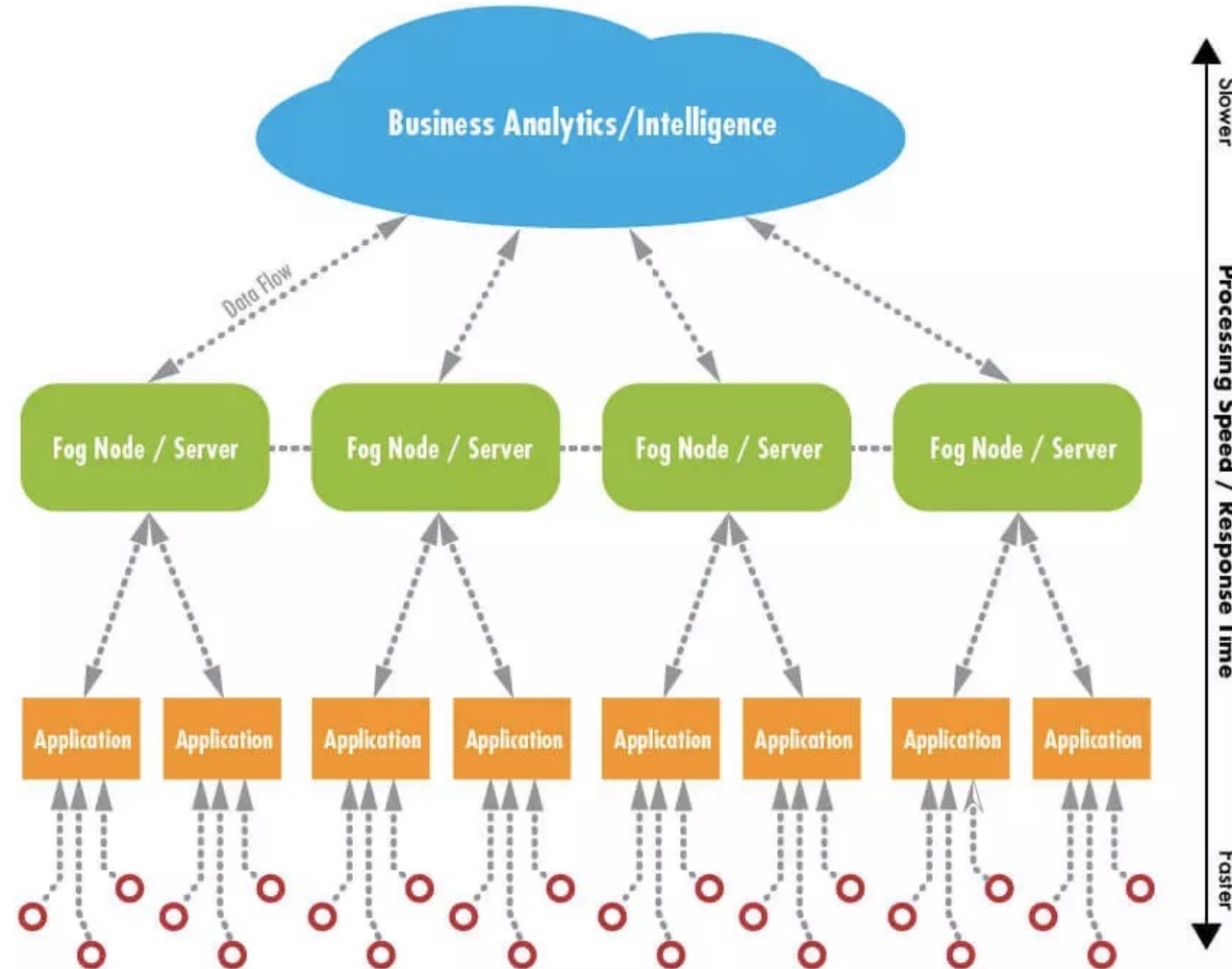
FOG LAYER

Local Network
Data Analysis & Reduction
Control Response
Virtualization/Standardization

EDGE LAYER

Large Volume Real-time Data Processing
At Source/On Premises Data Visualization
Industrial PCs
Embedded Systems
Gateways
Micro Data Storage

Sensors & Controllers (data origination)



[Source: <https://www.winsystems.com/cloud-fog-and-edge-computing-whats-the-difference/>]

Edge computing

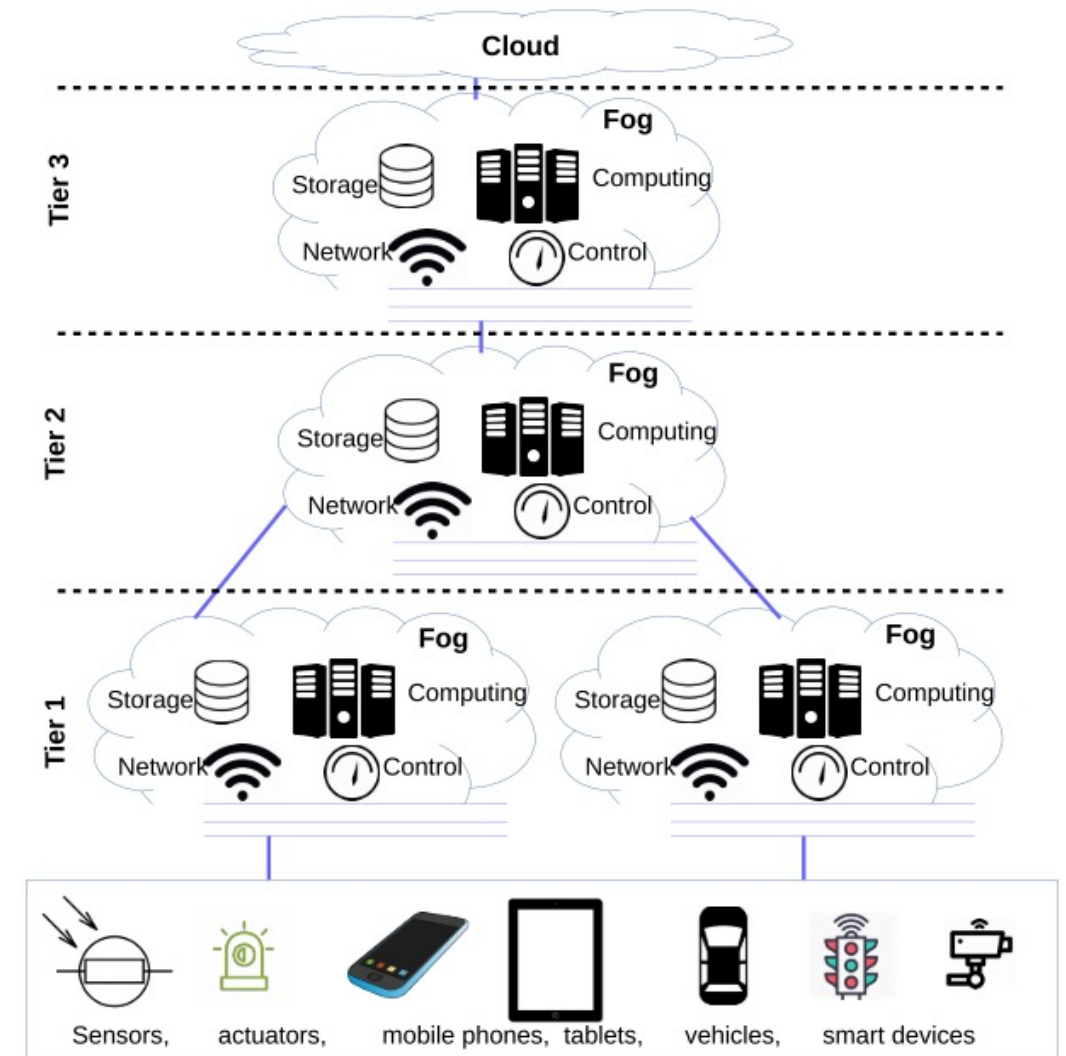
- Computation takes place at the edge of a device's network, which is known as edge computing
- A computer is connected with the network of the device, which processes the data and sends the data to the cloud in real-time
- That computer is known as “edge computer” or “edge node”
- Data is processed and transmitted to the devices instantly
- Edge nodes transmit all the data captured or generated by the device regardless of the importance of the data.

Fog computing

- Fog computing is an extension of cloud computing
- It is a layer in between the edge and the cloud
- When edge computers send huge amounts of data to the cloud, fog nodes receive the data and analyze what's important. Then the fog nodes transfer the important data to the cloud to be stored and delete the unimportant data or keep them with themselves for further analysis. In this way, fog computing saves a lot of space in the cloud and transfers important data quickly.

Fog (including Edge) computing

- Fog technology complements the role of cloud computing and distributes the data processing at the edge of the network, which provides faster responses to application queries and saves the network resources
- Fog computing model
 - Sensors
 - Actuators
 - Fog nodes at T1, T2, T3, etc. levels
 - Cloud
- Benefits of Fog computing
 - Move data to the best place for processing
 - Optimize latency
 - Conserve network bandwidth
 - Collect and secure data



Cloud is the centralized storage situated further from the endpoints than any other type of storage. This explains the highest latency, bandwidth cost, and network requirements. On the other hand, cloud is a powerful global solution that can handle huge amounts of data and scale effectively by engaging more computing resources and server space. It works great for big data analytics, long-term data storage and historical data analysis.

Fog acts as a middle layer between cloud and edge and provides the benefits of both. It relies on and works directly with the cloud handing out data that don't need to be processed on the go. At the same time, fog is placed closer to the edge. If necessary, it engages local computing and storage resources for real-time analytics and quick response to events.

Just like edge, fog is decentralized meaning that it consists of many nodes. However, unlike edge, fog has a network architecture. Fog nodes are connected with each other and can redistribute computing and storage to better solve given tasks.

Edge is the closest you can get to end devices, hence the lowest latency and immediate response to data. This approach allows to perform computing and store some (only limited) volume of data directly on devices, applications and edge gateways. It usually has a loosely connected structure where edge nodes work with data independently. This is what differentiates edge from network-based fog.

Here's a cloud vs. fog vs. edge computing comparison chart that gives a quick overview of these and other differences between these approaches.

Cloudlet

- A cloudlet is a mobility-enhanced small-scale cloud datacenter that is located at the edge of the Internet
- The main purpose of the cloudlet is supporting resource-intensive and interactive mobile applications by providing powerful computing resources to mobile devices with lower latency
- It is a new architectural element that extends today's cloud computing infrastructure
- It represents the middle tier of a 3-tier hierarchy: mobile device - cloudlet - cloud.

